Table of contents

01 → Neural Networks

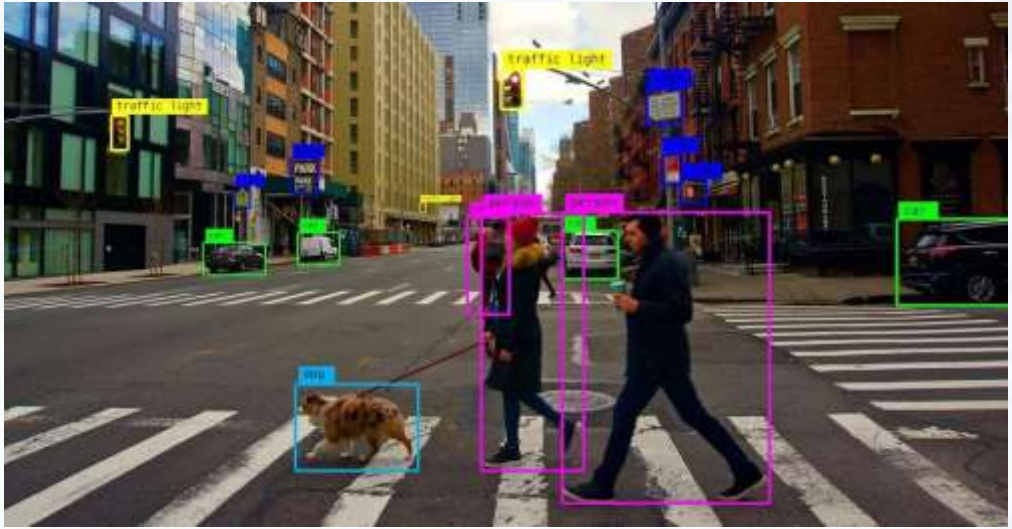
02 → Introduction to TensorFlow

01 →

Neural Networks

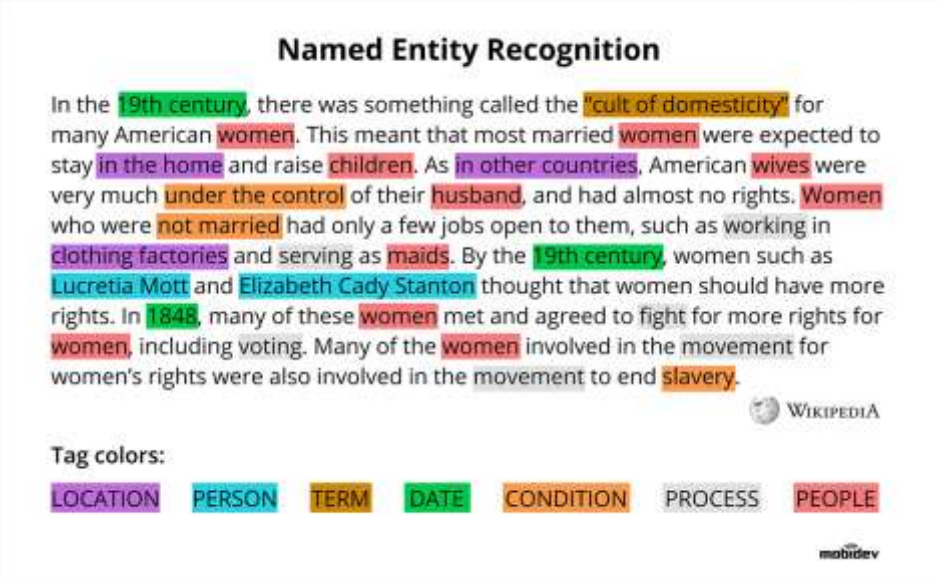
(AI)

Key applications



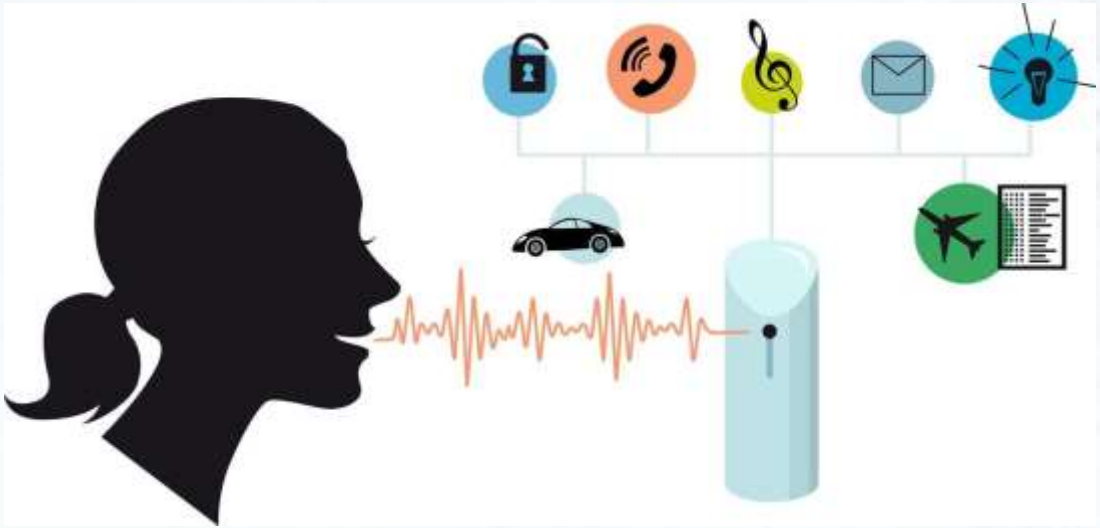
Computer Vision

Key applications



Natural Language Processing

Key applications



Speech Recognition

EARLY NEURAL NETWORKS

- Inspired by biology
- Created in the 1950's
- Outclassed by Von Neumann Architecture

DATA

- Networks need a lot of information to learn from
- The digital era and the internet has supplied that data

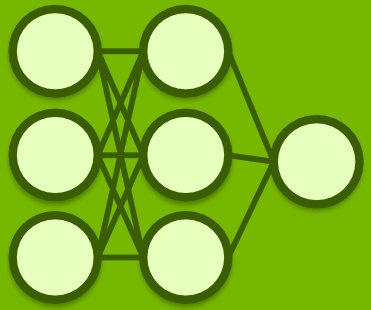
COMPUTING POWER

Need a way for our artificial “brain” to observe lots of data within a practical amount of time.

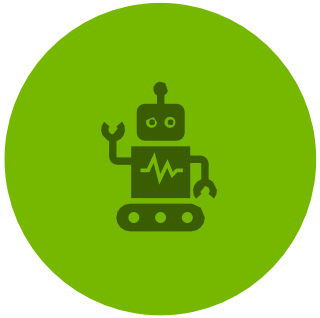
THE IMPORTANCE OF THE GPU

A Rendered Image

A Neural Network



COMPUTER VISION



ROBOTICS AND MANUFACTURING

OBJECT DETECTION

SELF DRIVING CARS

NATURAL LANGUAGE PROCESSING

REAL TIME TRANSLATION

VOICE RECOGNITION

VIRTUAL ASSISTANTS

RECOMMENDER SYSTEMS

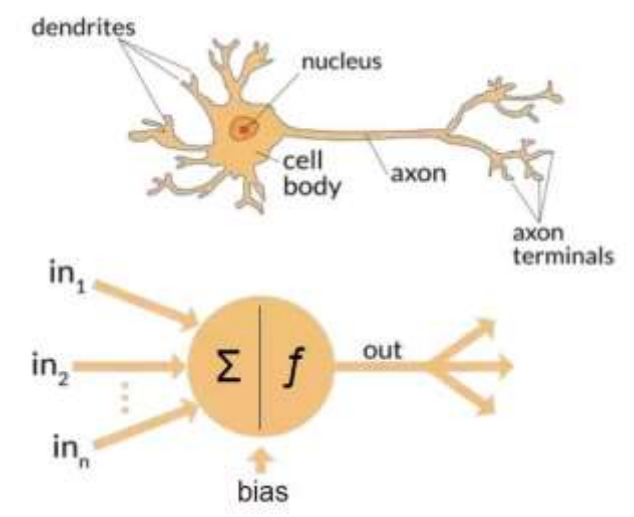
CONTENT CURATION

TARGETED ADVERTISING

SHOPPING RECOMMENDATIONS

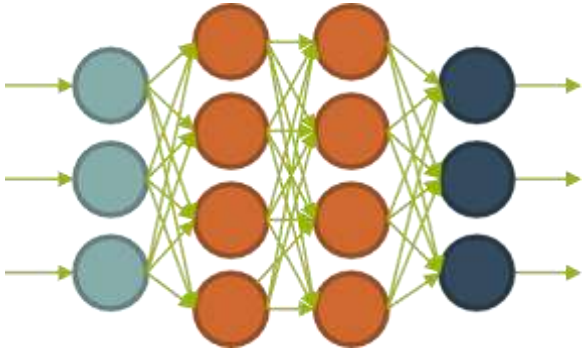
Biological Inspiration

- **Layers of neurons** form networks akin to how brain regions process signals hierarchically
- Artificial neurons mimic biological neurons: each computes a **weighted sum** of inputs, applies an **activation**, and propagates an output

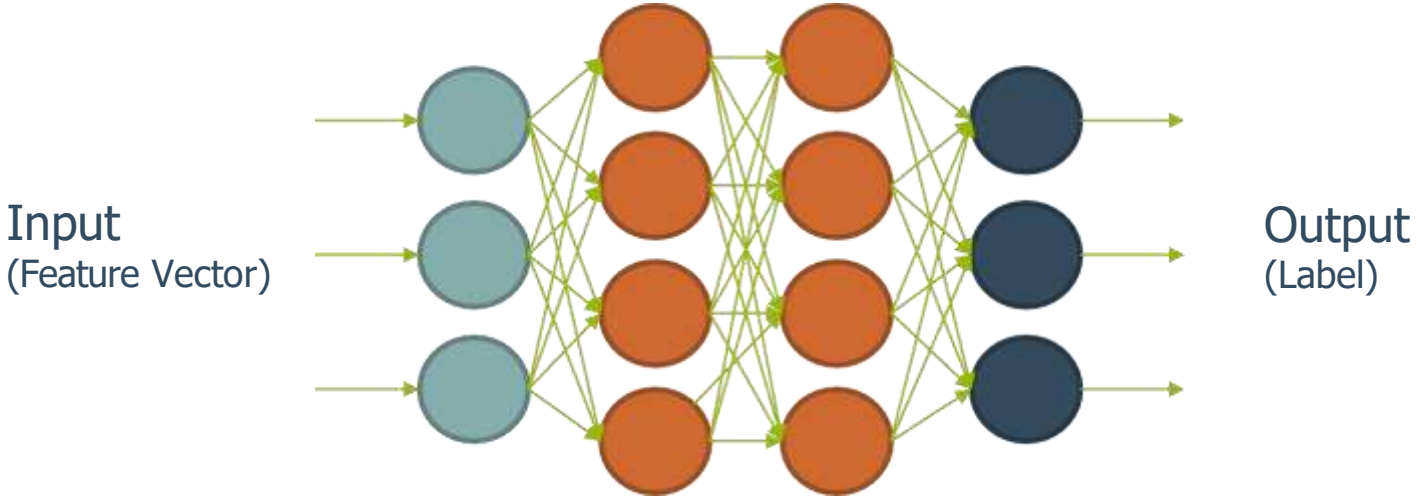


Motivation for Neural Nets

- Use biology as inspiration for mathematical model
- Get signals from previous neurons
- Generate signals (or not) according to inputs
- Pass signals on to next neurons
- By layering many neurons, can create complex model

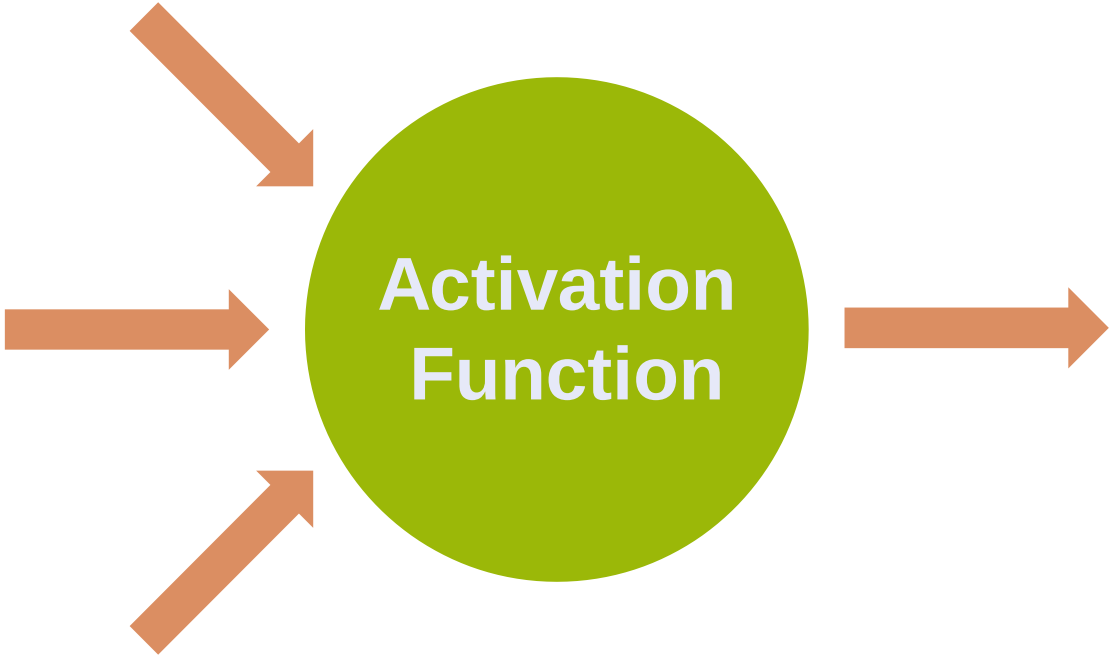


Neural Net Structure

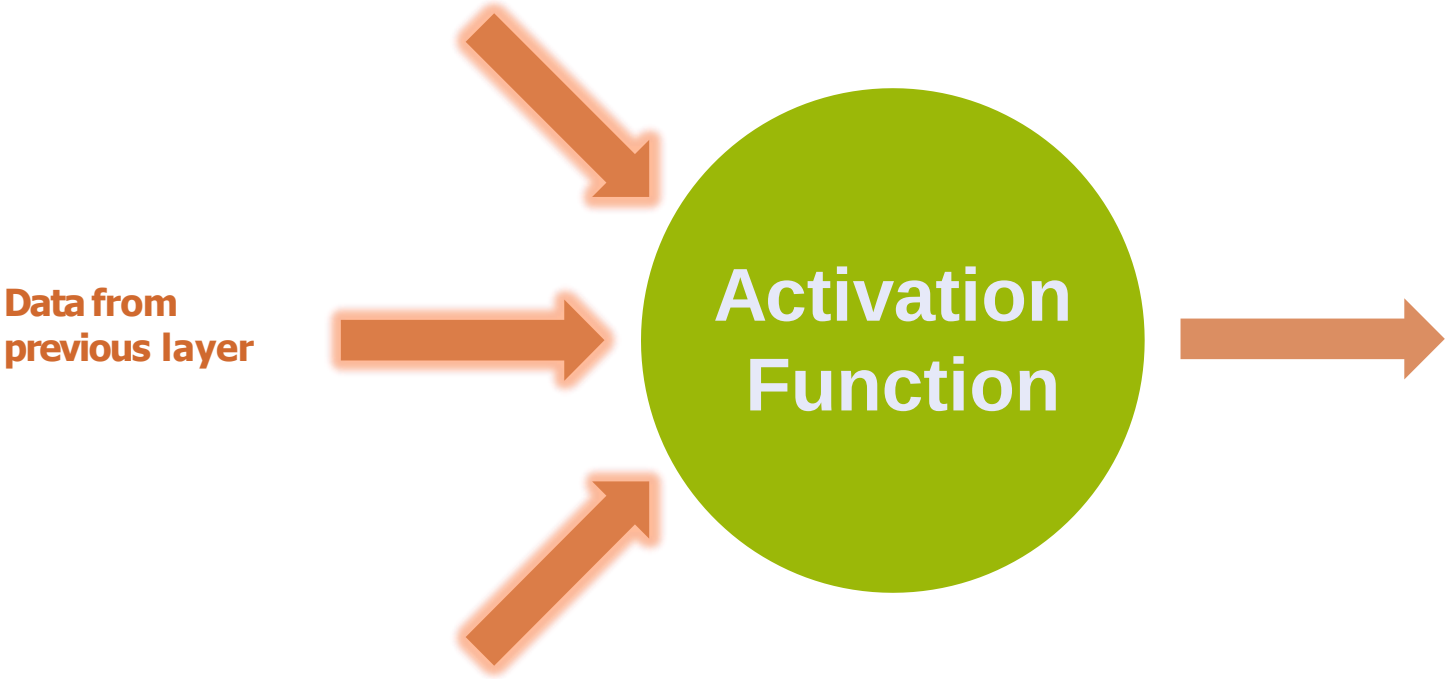


- Can think of it as a complicated computation engine
- We will ”train it” using our training data
- Then (hopefully) it will give good answers on new data

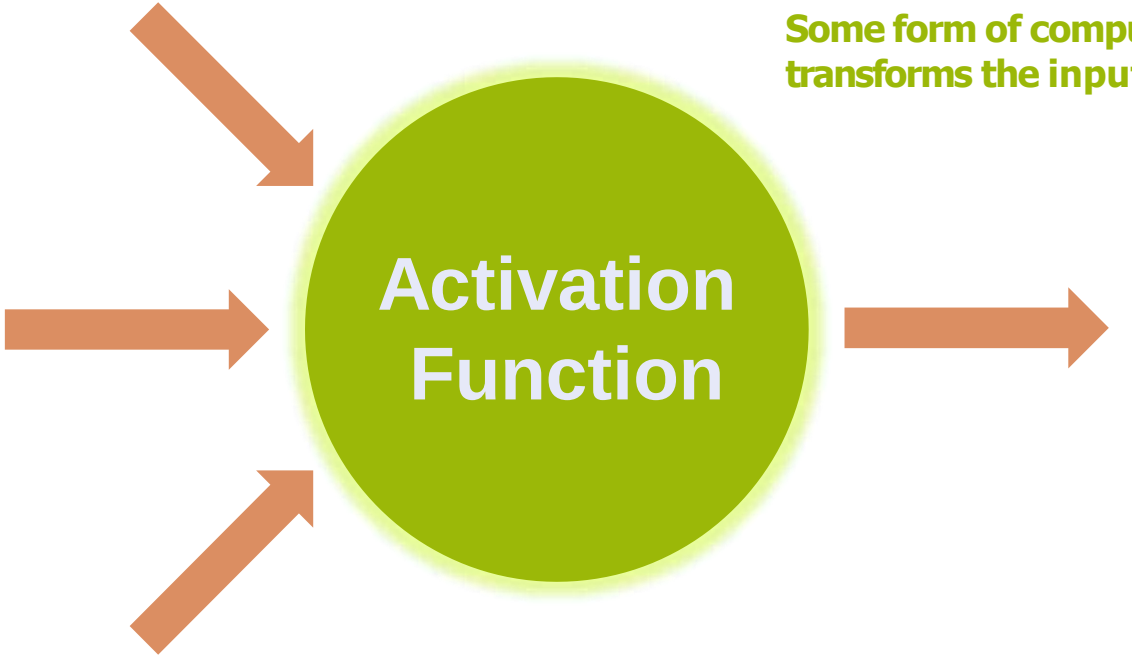
Basic Neuron Visualization



Basic Neuron Visualization

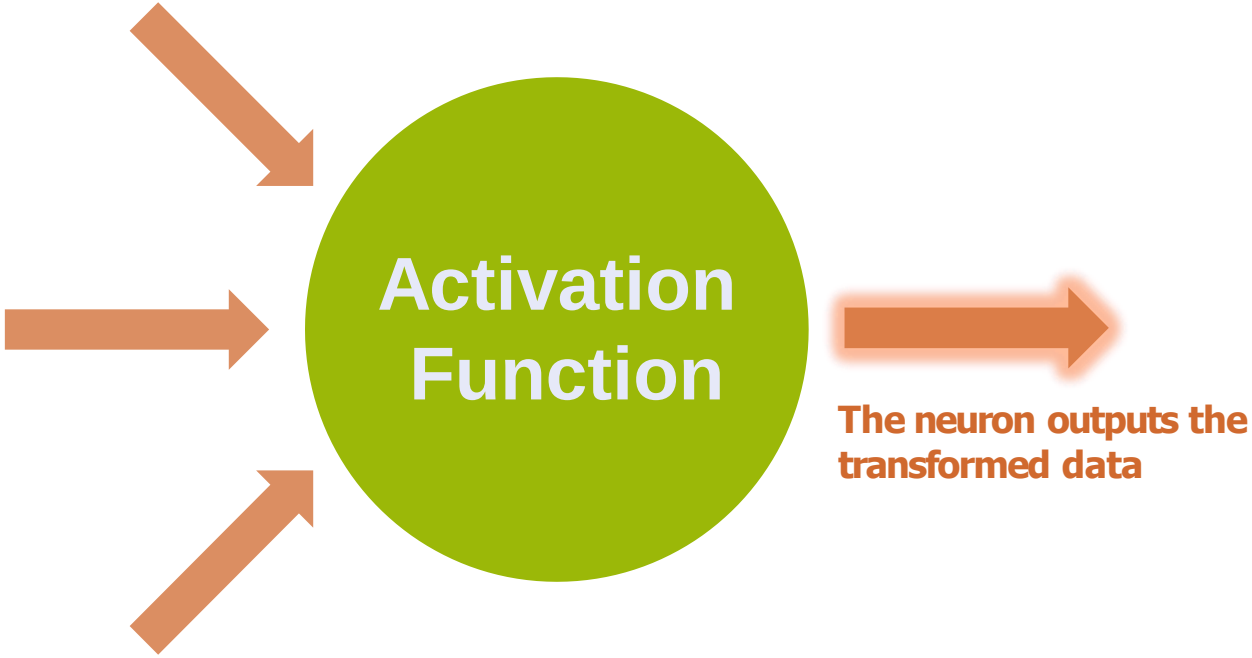


Basic Neuron Visualization

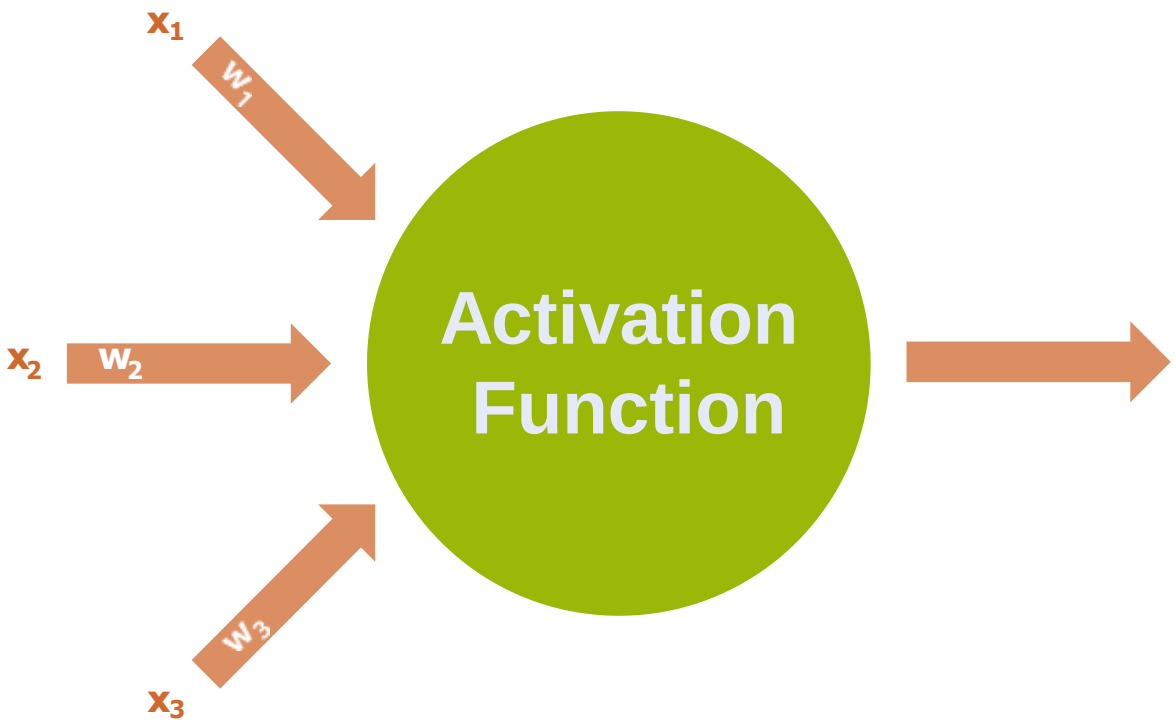


Some form of computation transforms the inputs

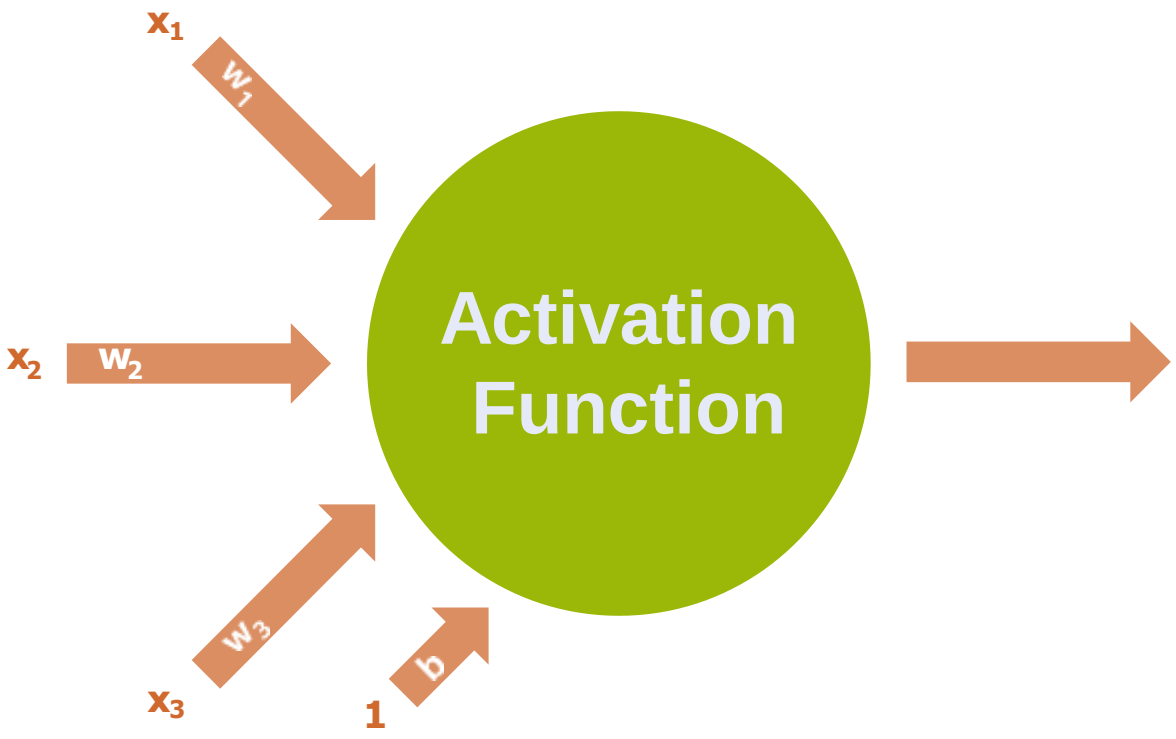
Basic Neuron Visualization



Basic Neuron Visualization

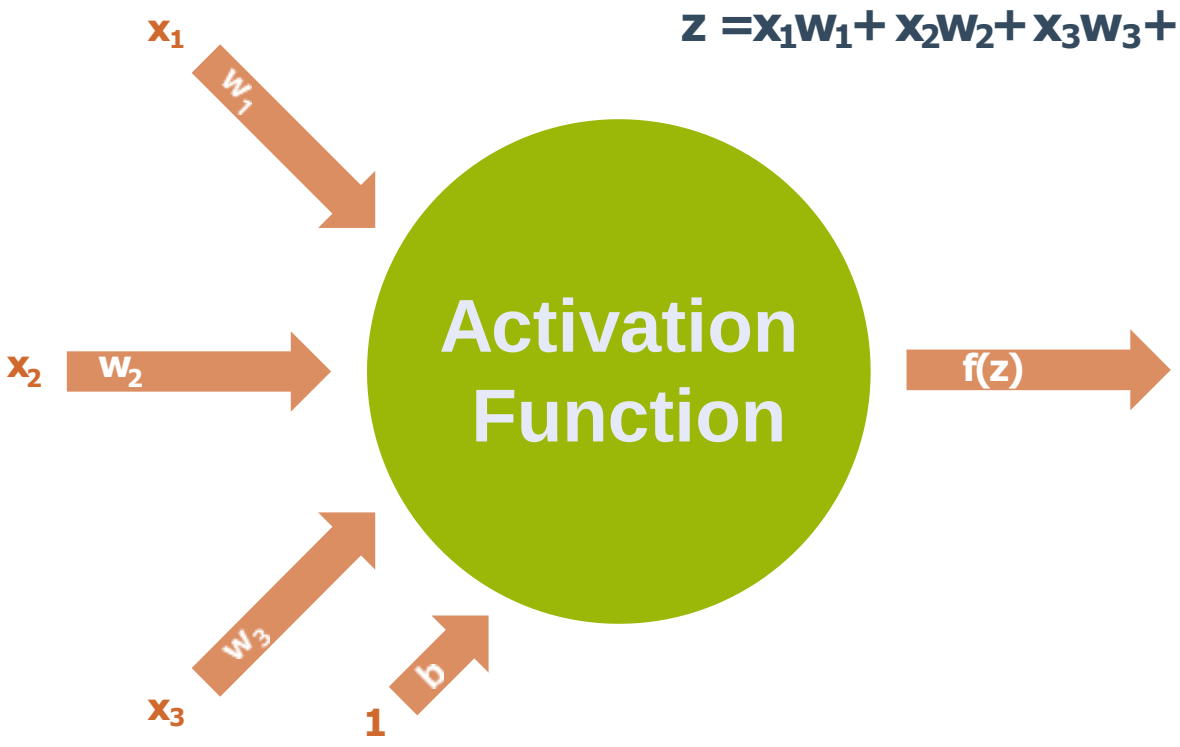


Basic Neuron Visualization



Basic Neuron Visualization

$$z = x_1w_1 + x_2w_2 + x_3w_3 + b$$



In Vector Notation

z =“net input”

b =“bias term”

f =activation function

a =output to next layer

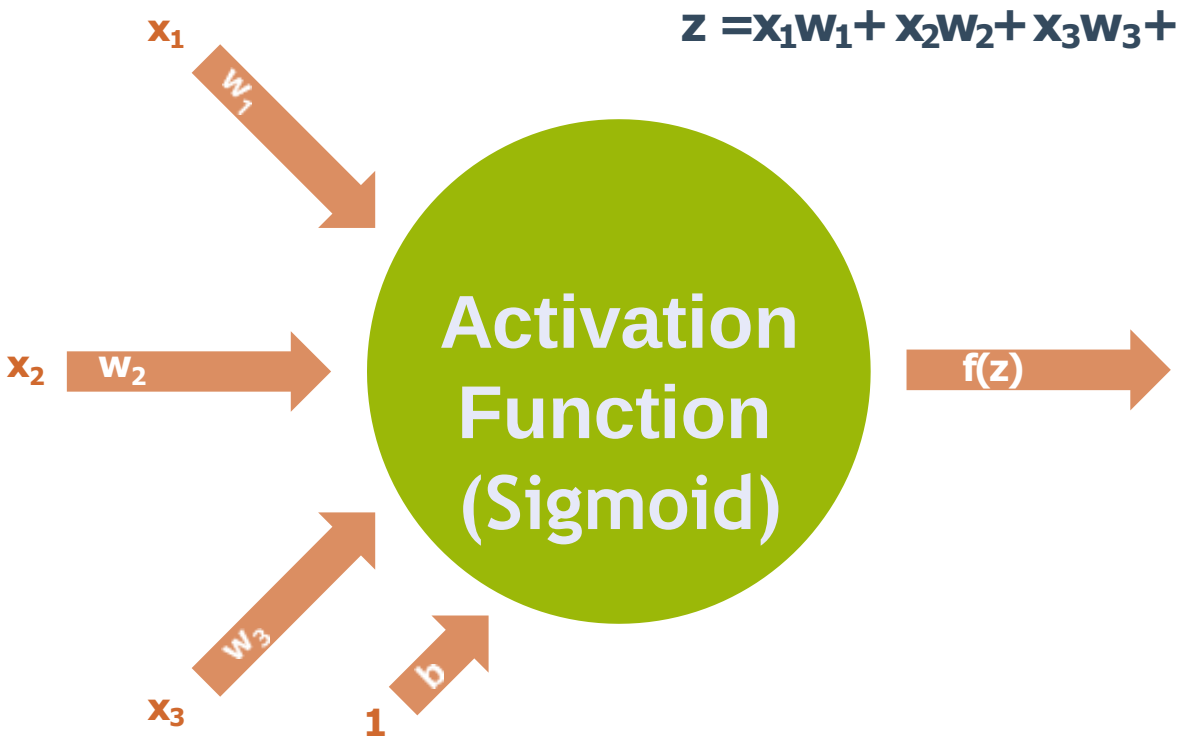
$$z = b + \sum_{i=1}^{m} x_i w_i$$

$$z = b + x^T w$$

$$a = f(z)$$

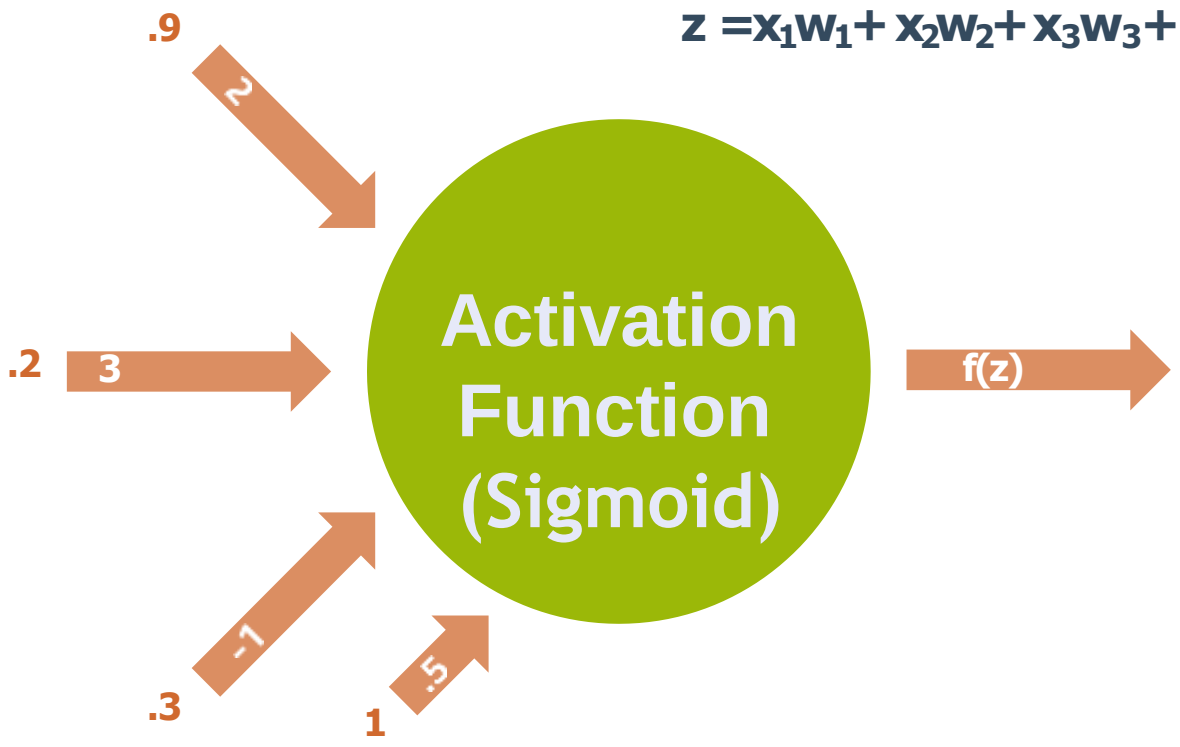
Example Neuron Computation

$$z = x_1w_1 + x_2w_2 + x_3w_3 + b$$



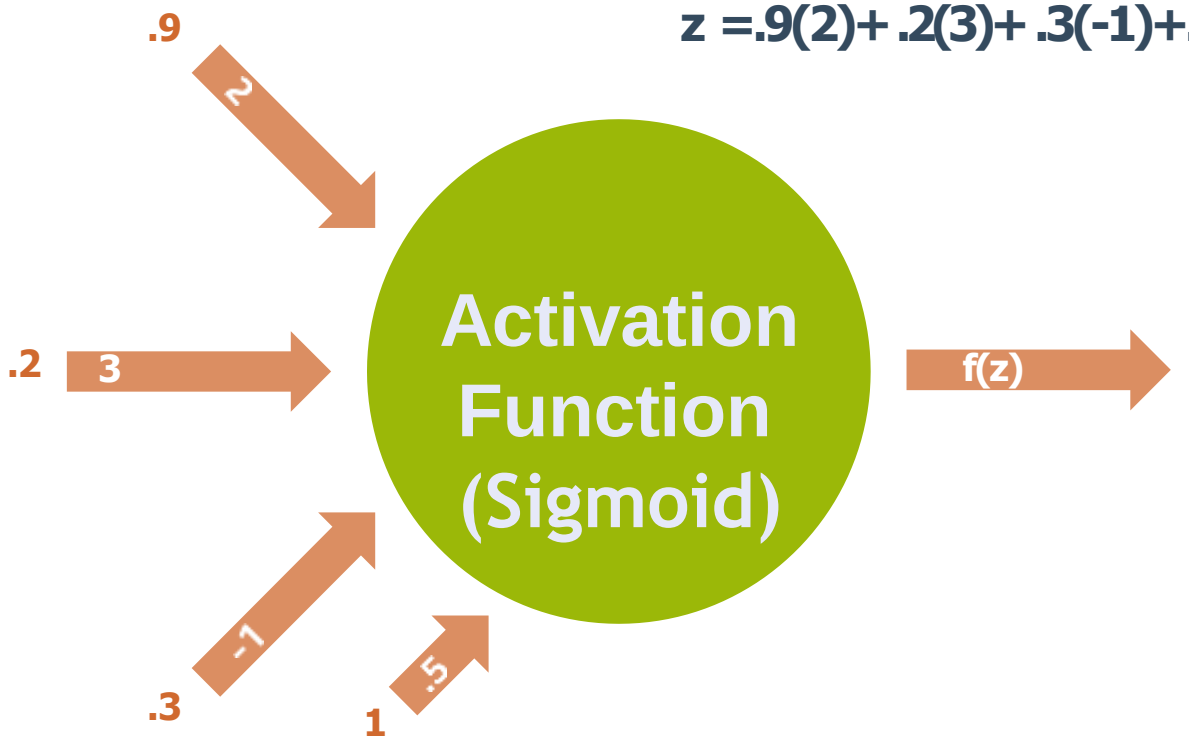
Example Neuron Computation

$$z = x_1w_1 + x_2w_2 + x_3w_3 + b$$

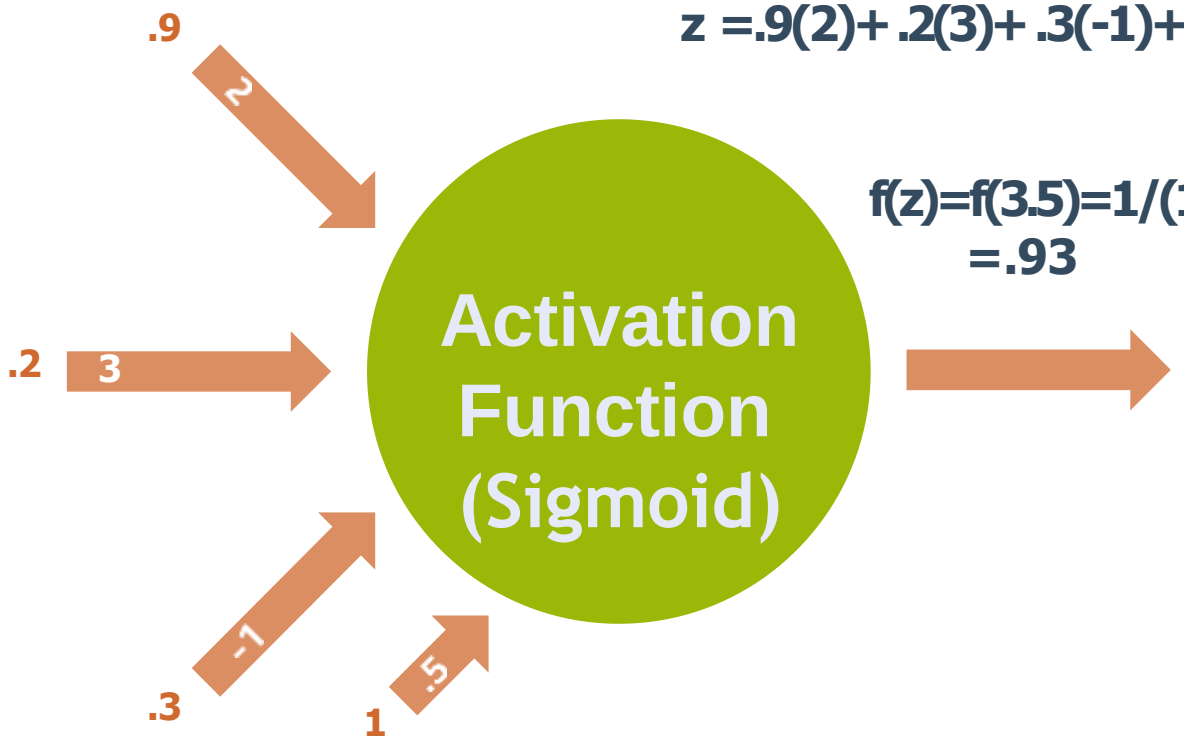


Example Neuron Computation

$$z = .9(2) + .2(3) + .3(-1) + .5 = 2.6$$



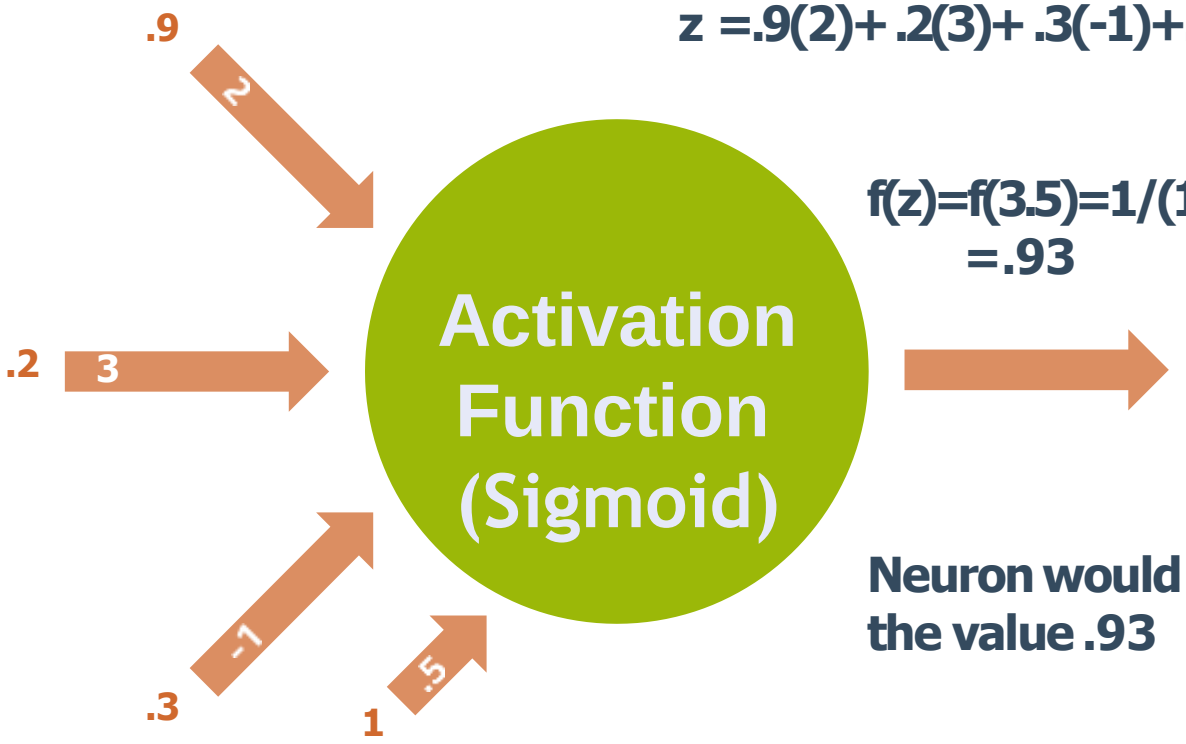
Example Neuron Computation



$$z = .9(2) + .2(3) + .3(-1) + .5 = 2.6$$

$$f(z) = f(3.5) = 1/(1+\exp(-2.6)) = .93$$

Example Neuron Computation



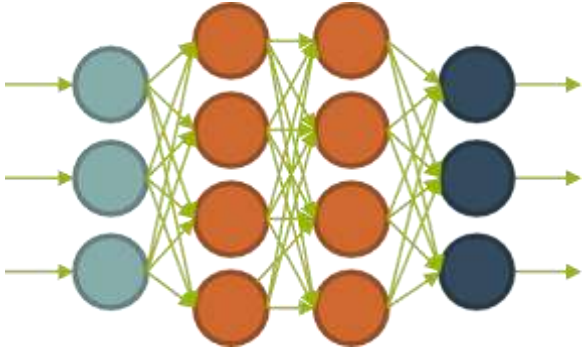
$$z = .9(2) + .2(3) + .3(-1) + .5 = 2.6$$

$$f(z) = f(3.5) = 1/(1+\exp(-2.6)) = .93$$

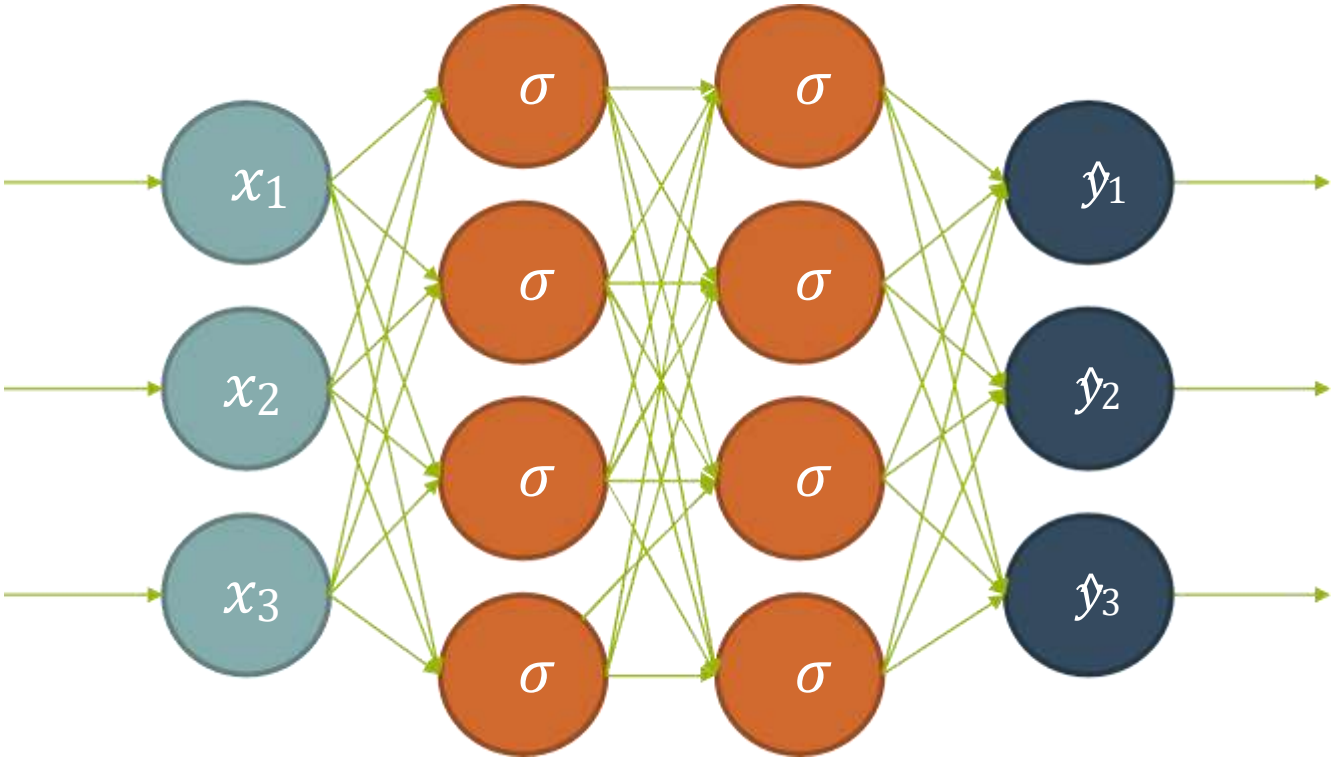
Neuron would output the value .93

Why Neural Nets?

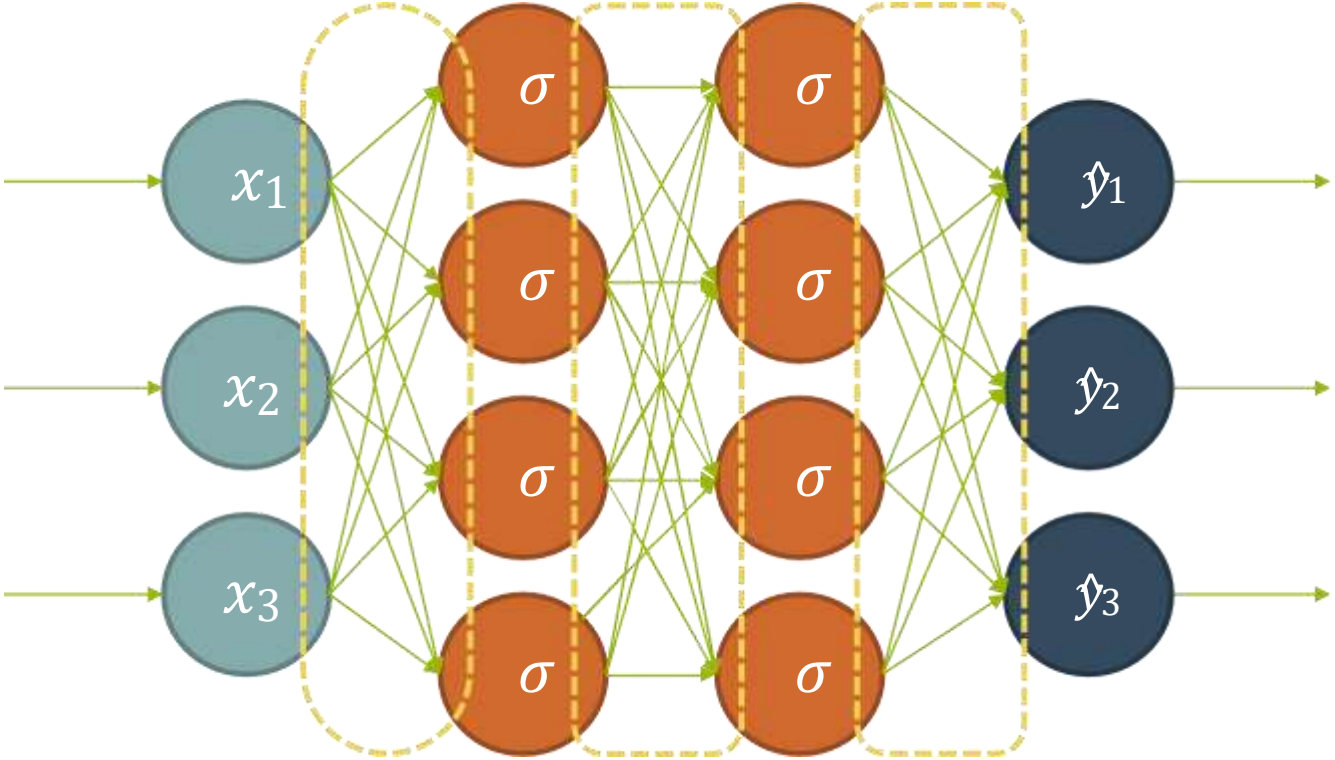
- Why not just use a single neuron? Why do we need a larger network?
- A single neuron (like logistic regression) only permits a linear decision boundary.
- Most real-world problems are considerably more complicated!



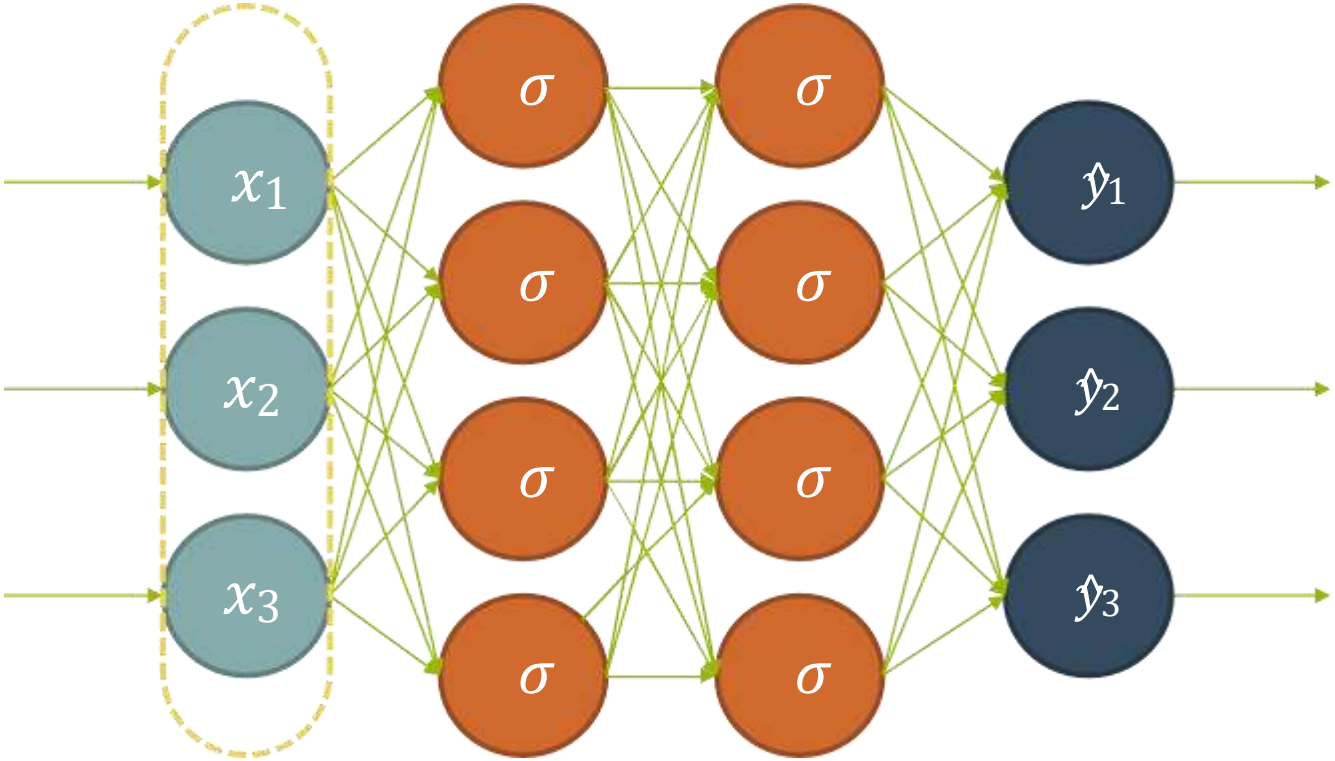
Feedforward Neural Network



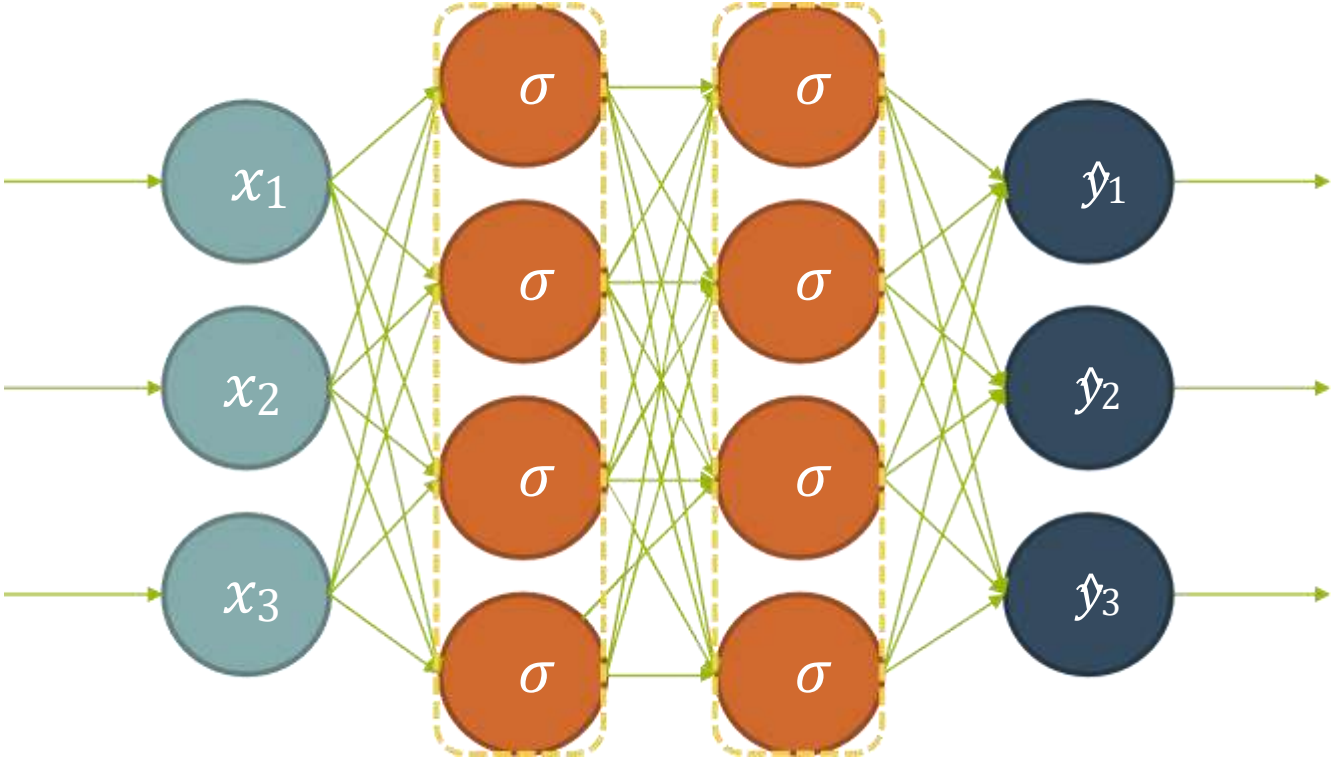
Weights


x_1
x_2
x_3
σ
σ
σ
σ
σ
σ
σ
σ
$\hat{y}_1$
$\hat{y}_2$
$\hat{y}_3$

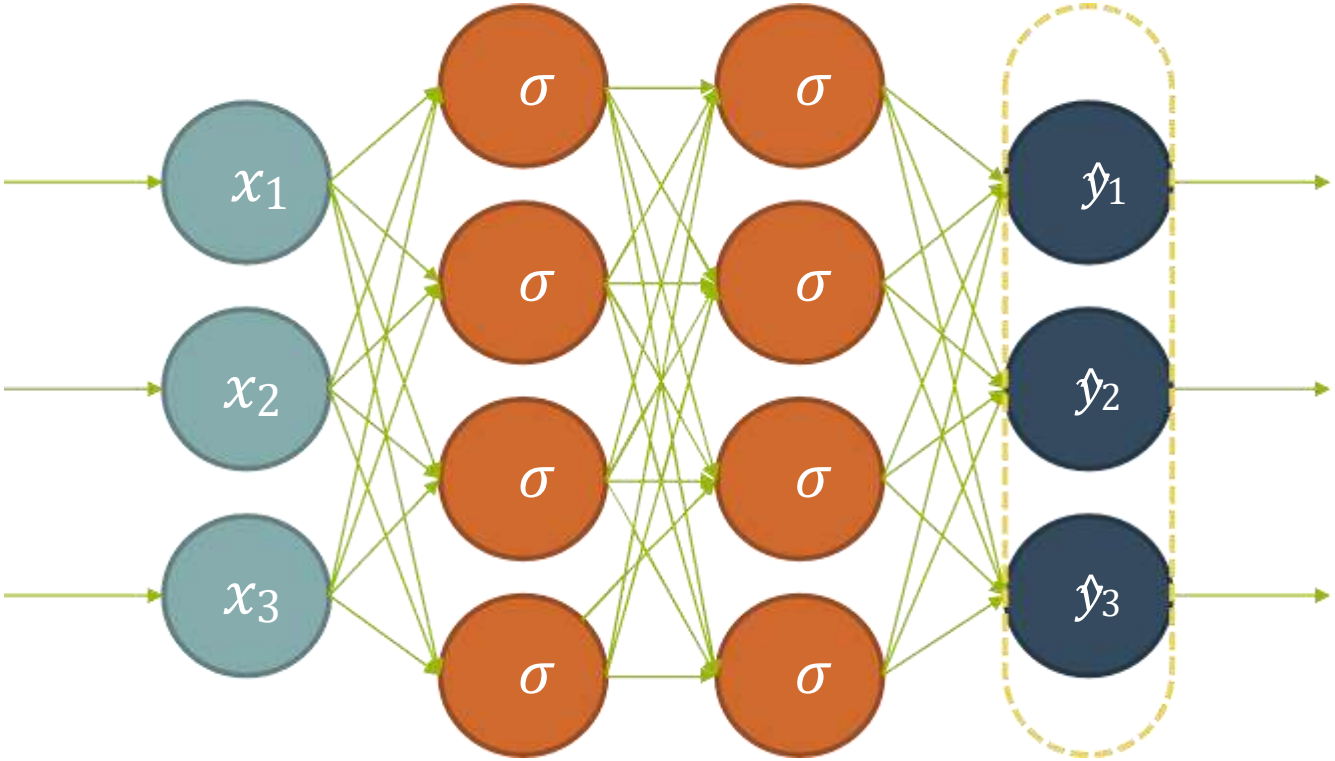
Input Layer


x_1
x_2
x_3
σ
σ
σ
σ
σ
σ
σ
σ
$\hat{y}_1$
$\hat{y}_2$
$\hat{y}_3$

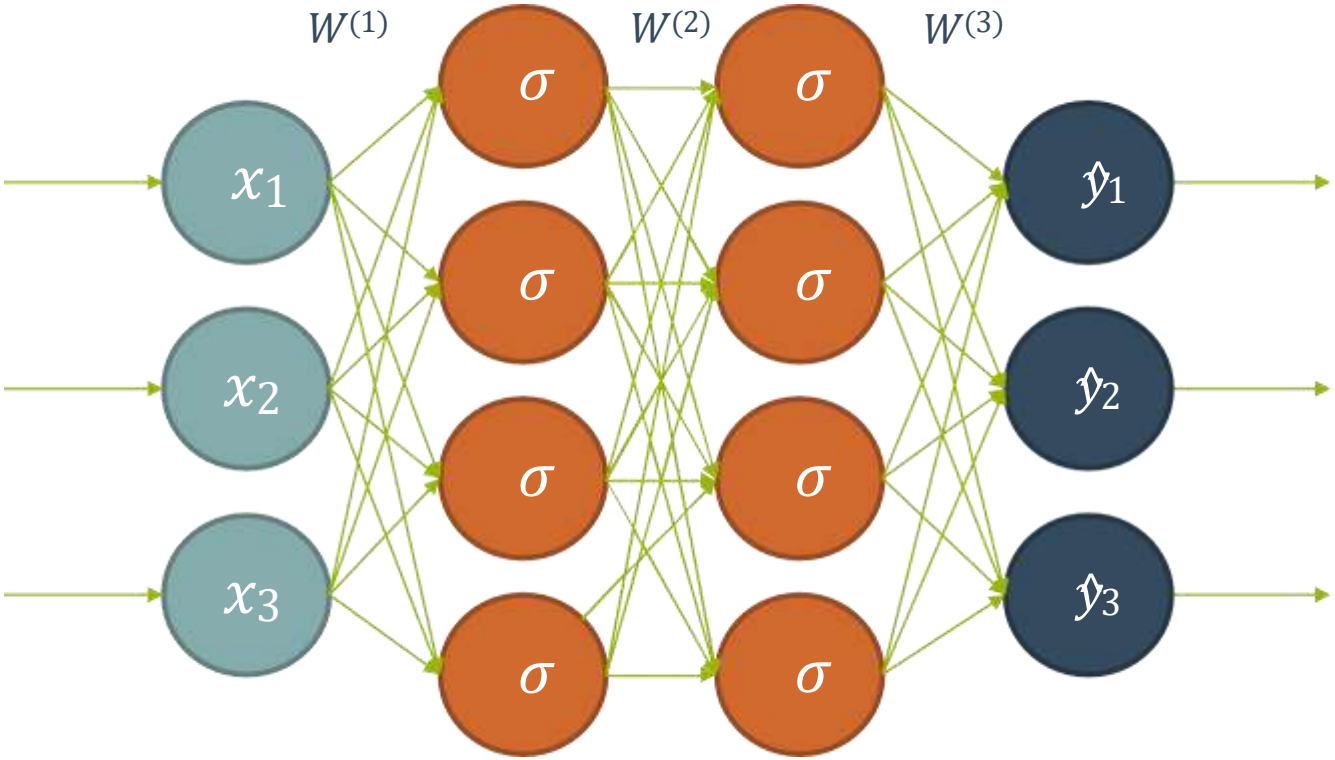
Hidden Layers



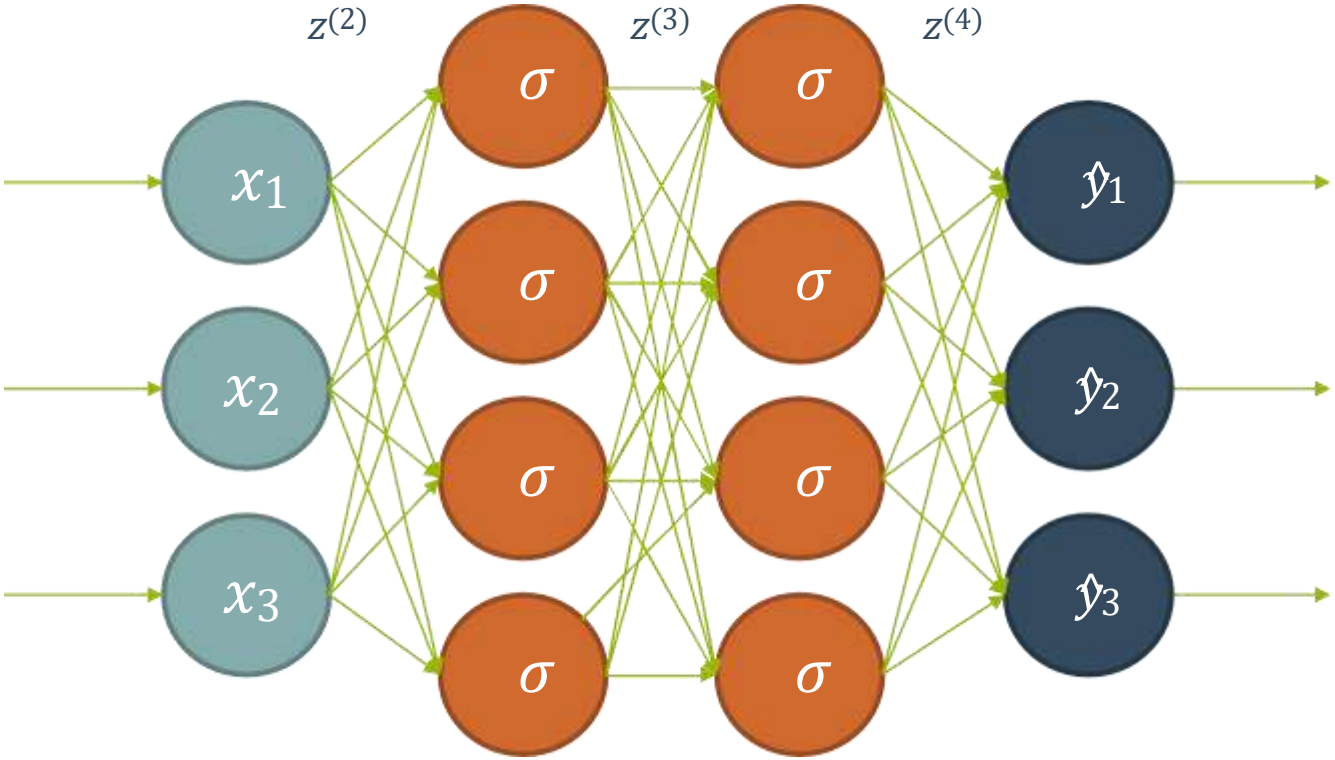
Output Layer



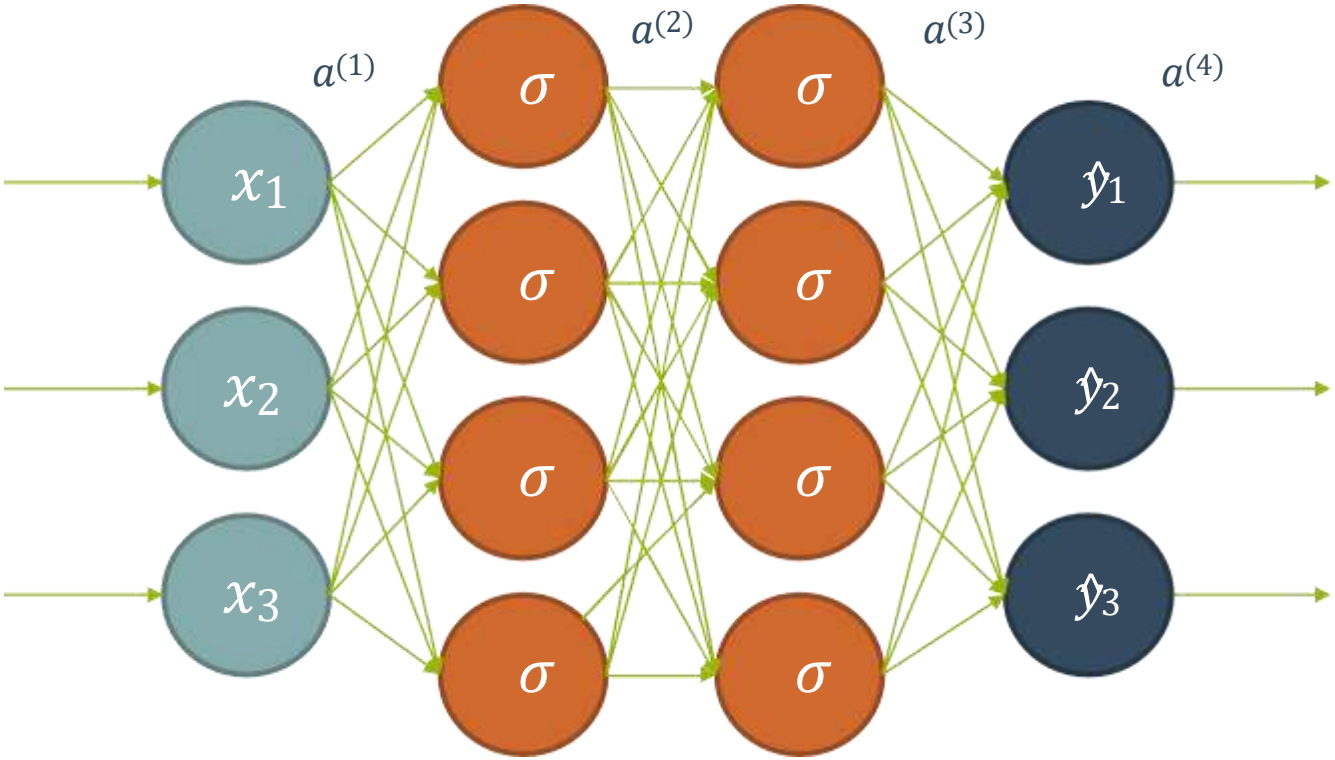
Weights (represented by matrices)



Net Input (sum of weighted inputs, before activation function)



Activations (output of neurons to next layer)



Multiple data points

- In practice, we do these computation for many data points at the same time,
- by "stacking" the rows into a matrix. But the equations look the same!

- Input: matrix x (an nx3 matrix)(each row a single instance)
- Output: vector $\hat{y}$ (an nx3 matrix) (each row a single prediction)

$$z^{(2)} = xW^{(1)} \qquad a^{(2)} = \sigma(z^{(2)})$$

$$z^{(3)} = a^{(2)}W^{(2)} \qquad a^{(3)} = \sigma(z^{(3)})$$

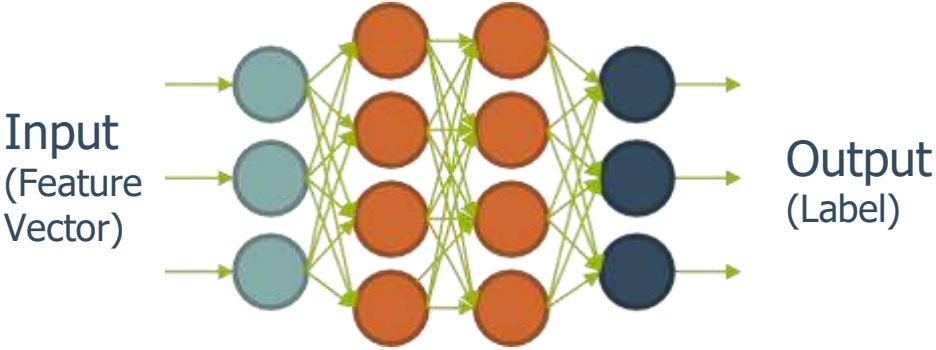
$$z^{(4)} = a^{(3)}W^{(3)} \qquad \hat{y} = softmax(z^{(4)})$$

How to Train a Neural Net?

- Put in training inputs, get the output
- Compare output to correct answers: look at loss function J
- Adjust and repeat!
- Backpropagation tells us how to make a single adjustment using calculus.

How to Train a Neural Net?

- Put in training inputs, get the output
- Compare output to correct answers: look at loss function J
- Adjust and repeat!
- Backpropagation tells us how to make a single adjustment using calculus.



How have we trained before?

- Gradient Descent!

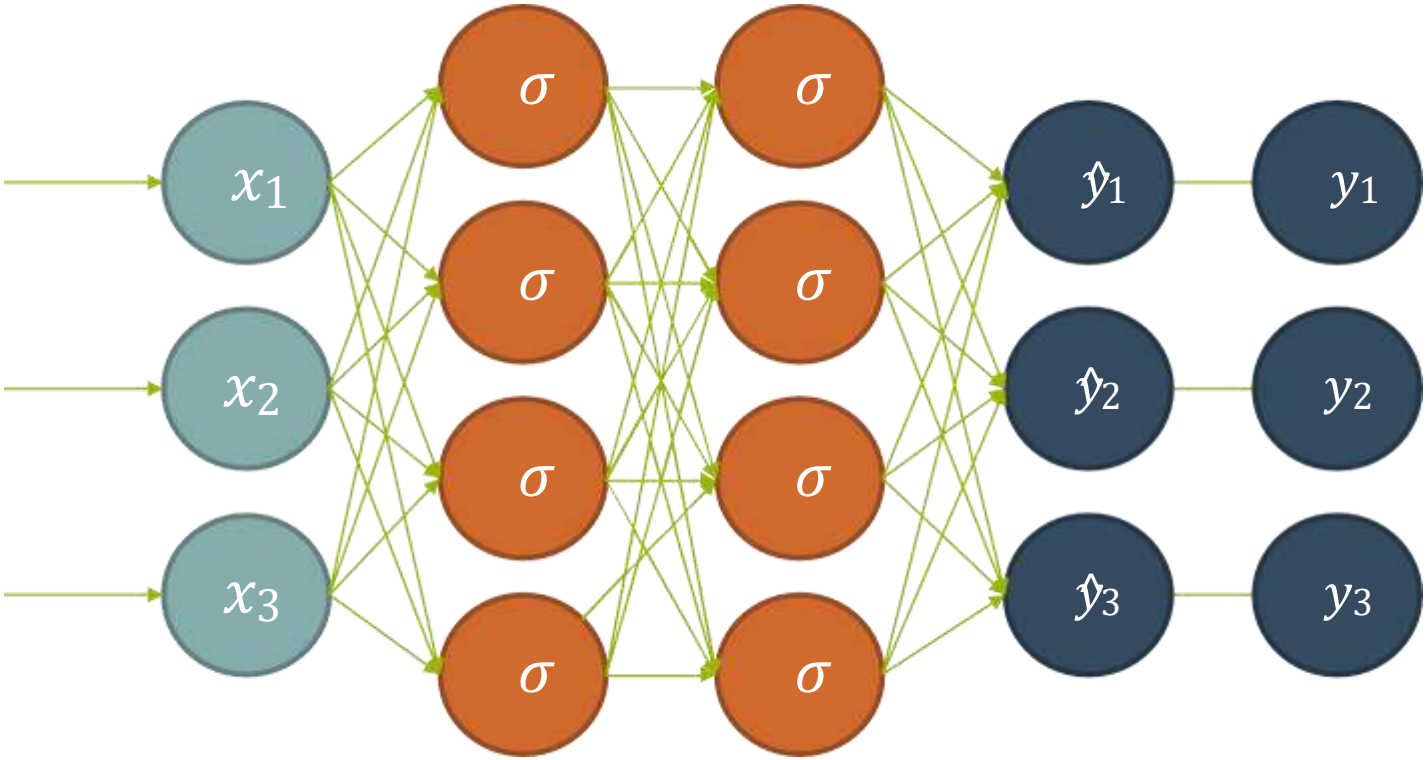
1. Make prediction
2. Calculate Loss
3. Calculate gradient of the loss function w.r.t. parameters
4. Update parameters by taking a step in the opposite direction
5. Iterate

How have we trained before?

- Gradient Descent!

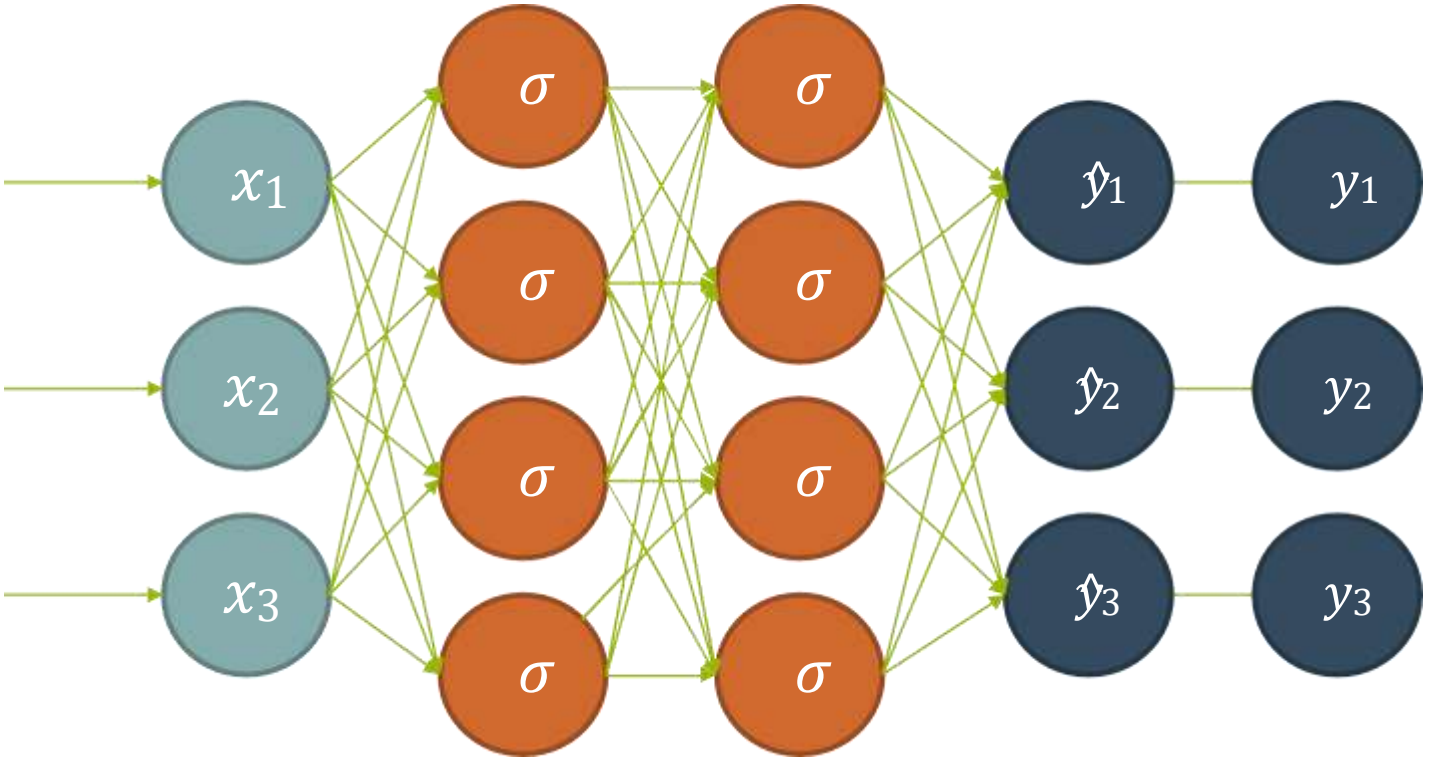
1. Make prediction
2. Calculate Loss
3. Calculate gradient of the loss function w.r.t. parameters
4. Update parameters by taking a step in the opposite direction
5. Iterate

Feedforward Neural Network



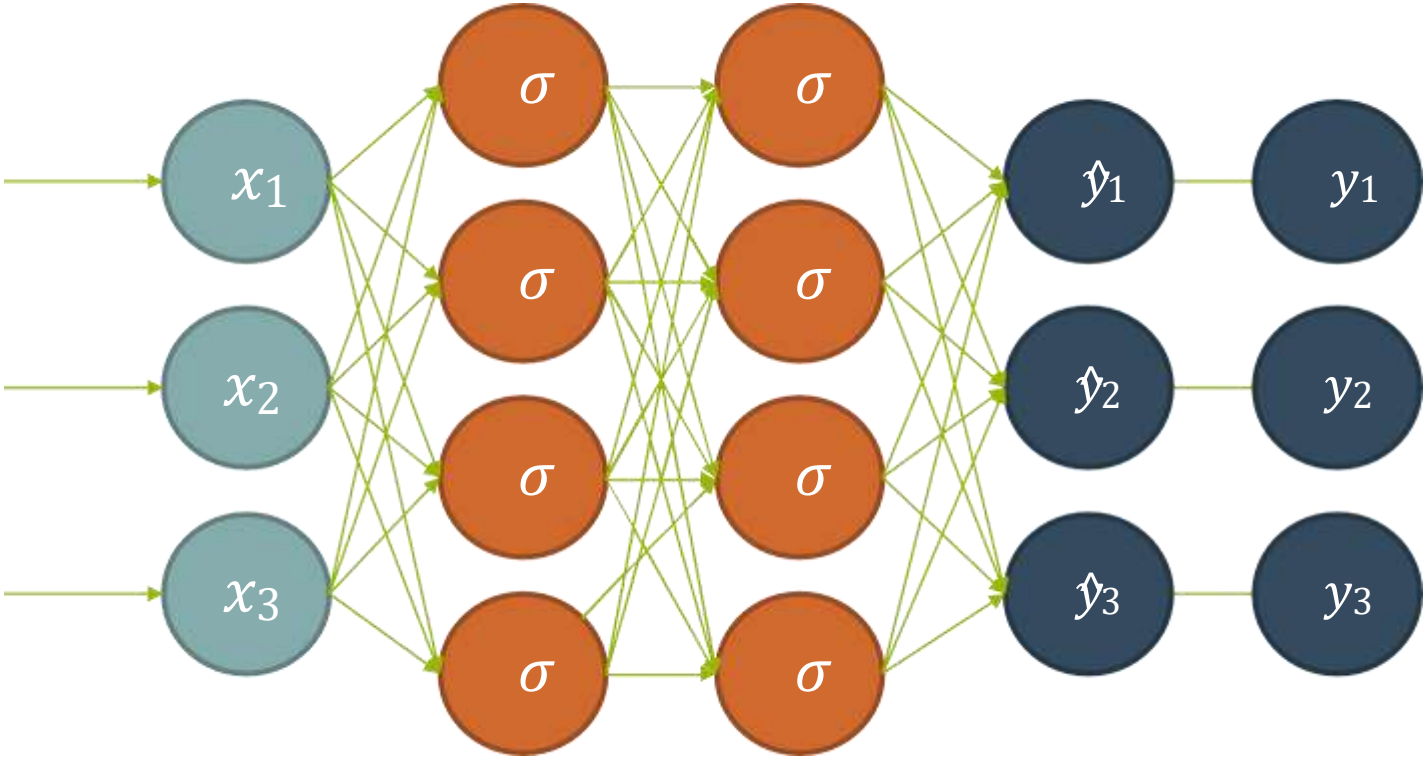
Forward Propagation

Pass in Input

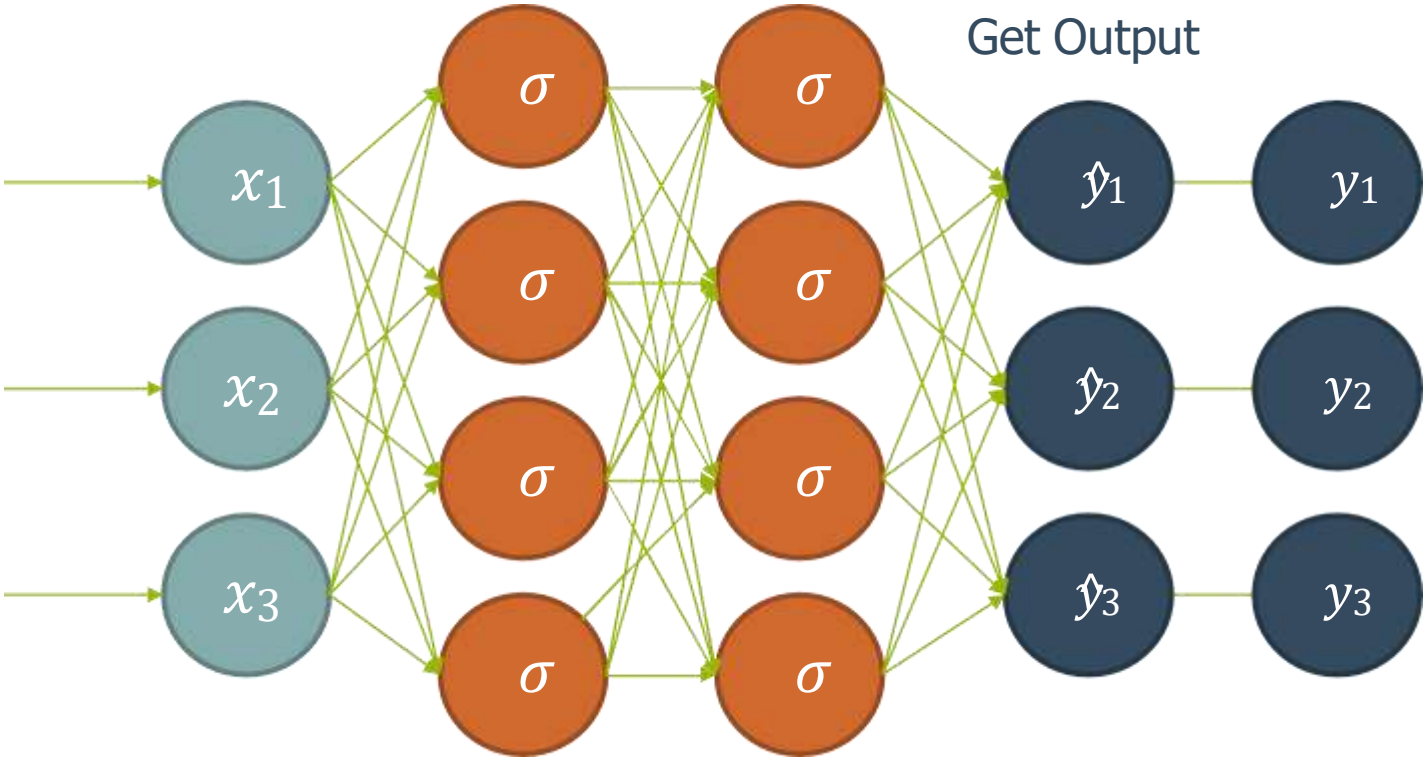


Forward Propagation

Calculate each Layer

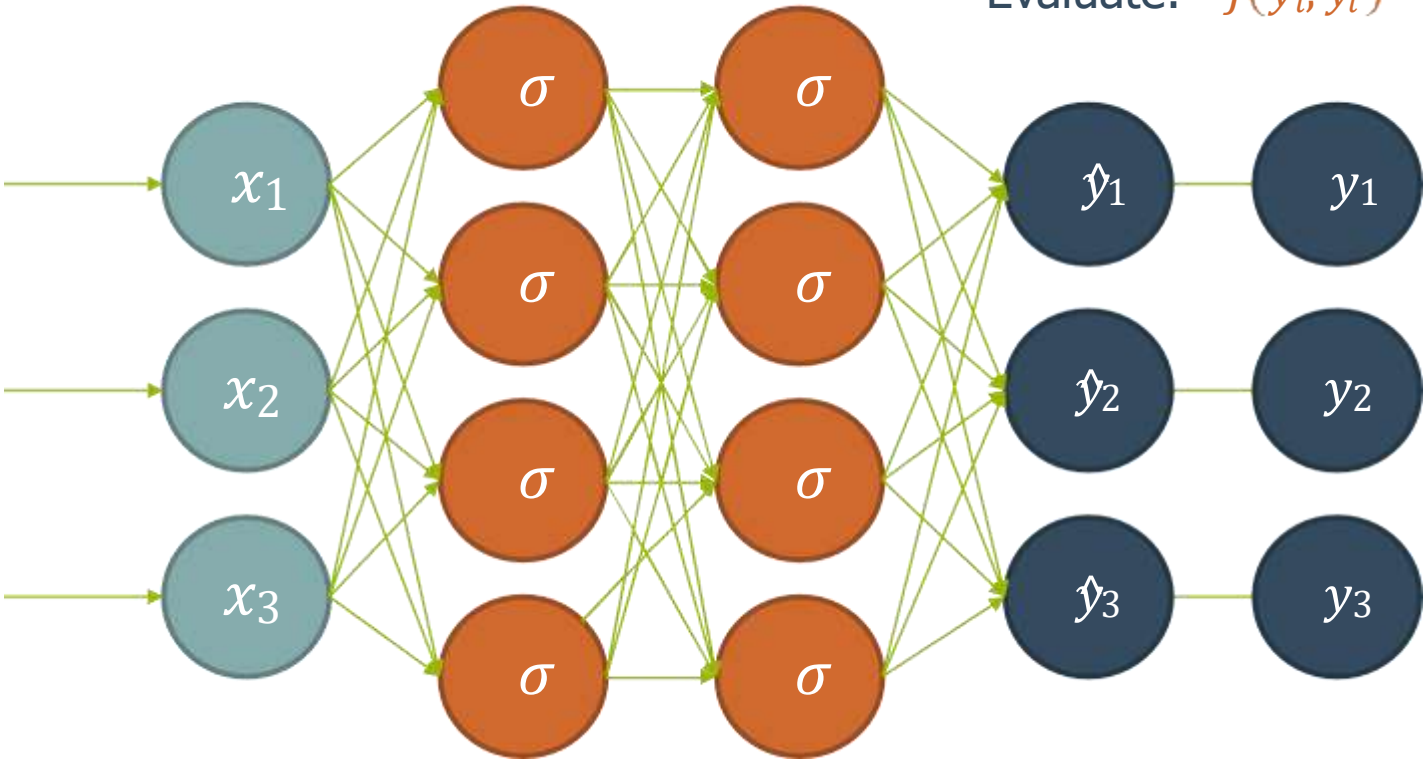


Forward Propagation



Forward Propagation

Evaluate: $J(y_i, \hat{y}_i)$



How have we trained before?

- Gradient Descent!

1. Make prediction
2. Calculate Loss
3. Calculate gradient of the loss function w.r.t. parameters
4. Update parameters by taking a step in the opposite direction
5. Iterate

How to calculate gradient?

Chain rule

How to Train a Neural Net?

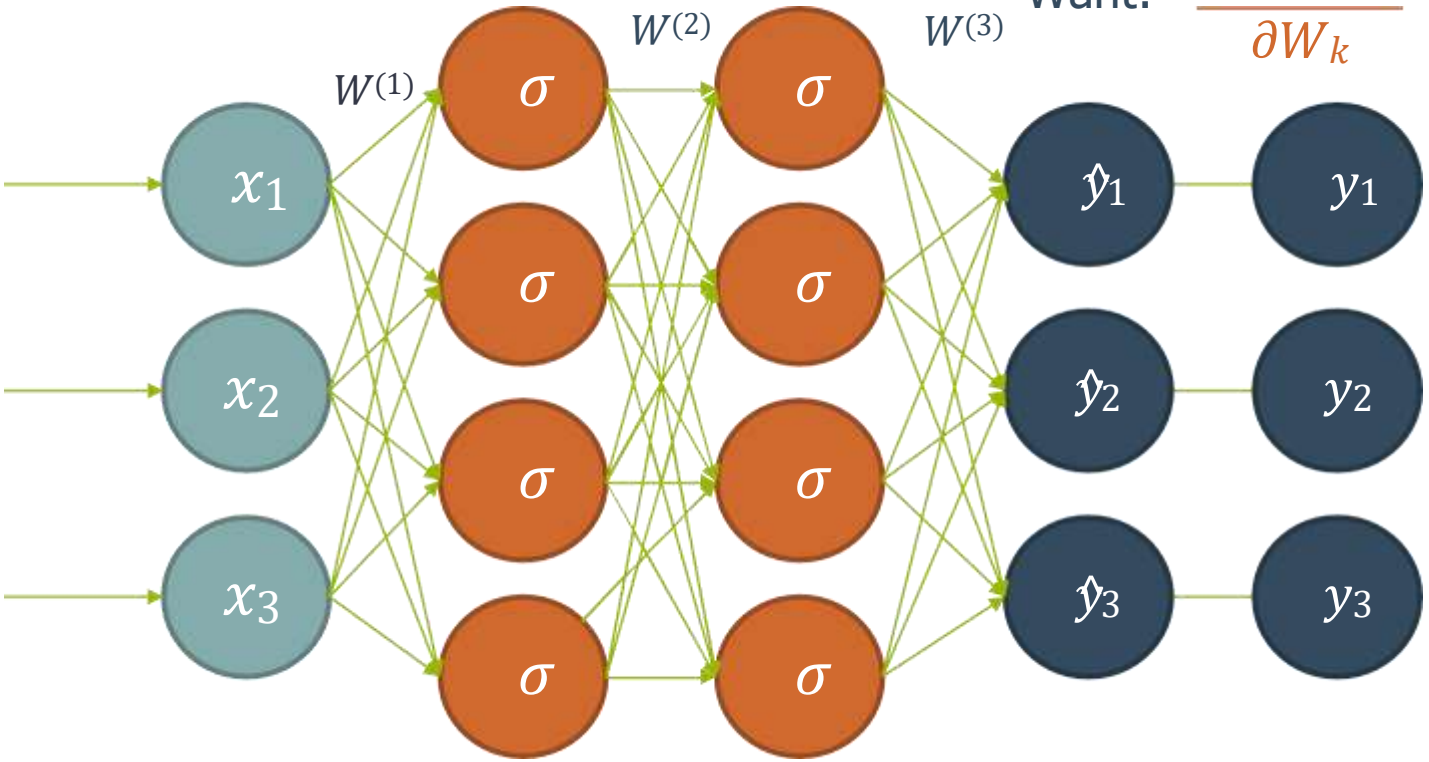
- How could we change the weights to make our Loss Function lower?
- Think of neural net as a function F: X -> Y
- F is a complex computation involving many weights W_k
- Given the structure, the weights "define" the function F (and therefore define our model)
- Loss Function is J(y,F(x))

How to Train a Neural Net?

- Get $\frac{\partial J}{\partial W_k}$ for every weight in the network.
- This tells us what direction to adjust each Wk if we want to lower our loss function.
- Make an adjustment and repeat!

Feedforward Neural Network

Want: $\frac{\partial J(y_i, \hat{y}_i)}{\partial W_k}$



Punchline

$$\frac{\partial J}{\partial W^{(3)}} = (\hat{y} - y) \cdot a^{(3)}$$

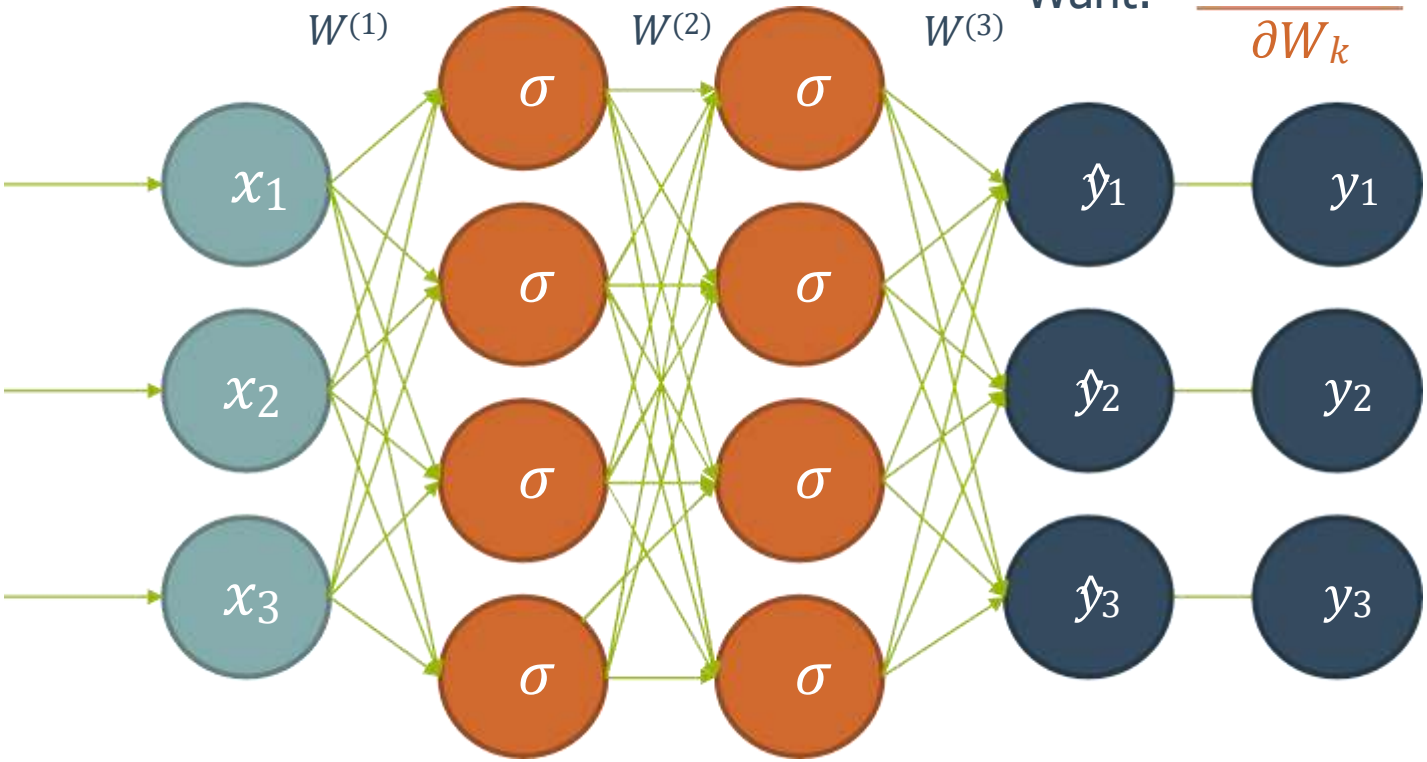
$$\frac{\partial J}{\partial W^{(2)}} = (\hat{y} - y) \cdot W^{(3)} \cdot \sigma'\left(z^{(3)}\right) \cdot a^{(2)}$$

$$\frac{\partial J}{\partial W^{(1)}} = (\hat{y} - y) \cdot W^{(3)} \cdot \sigma'\left(z^{(3)}\right) \cdot W^{(2)} \cdot \sigma'\left(z^{(2)}\right) \cdot X$$

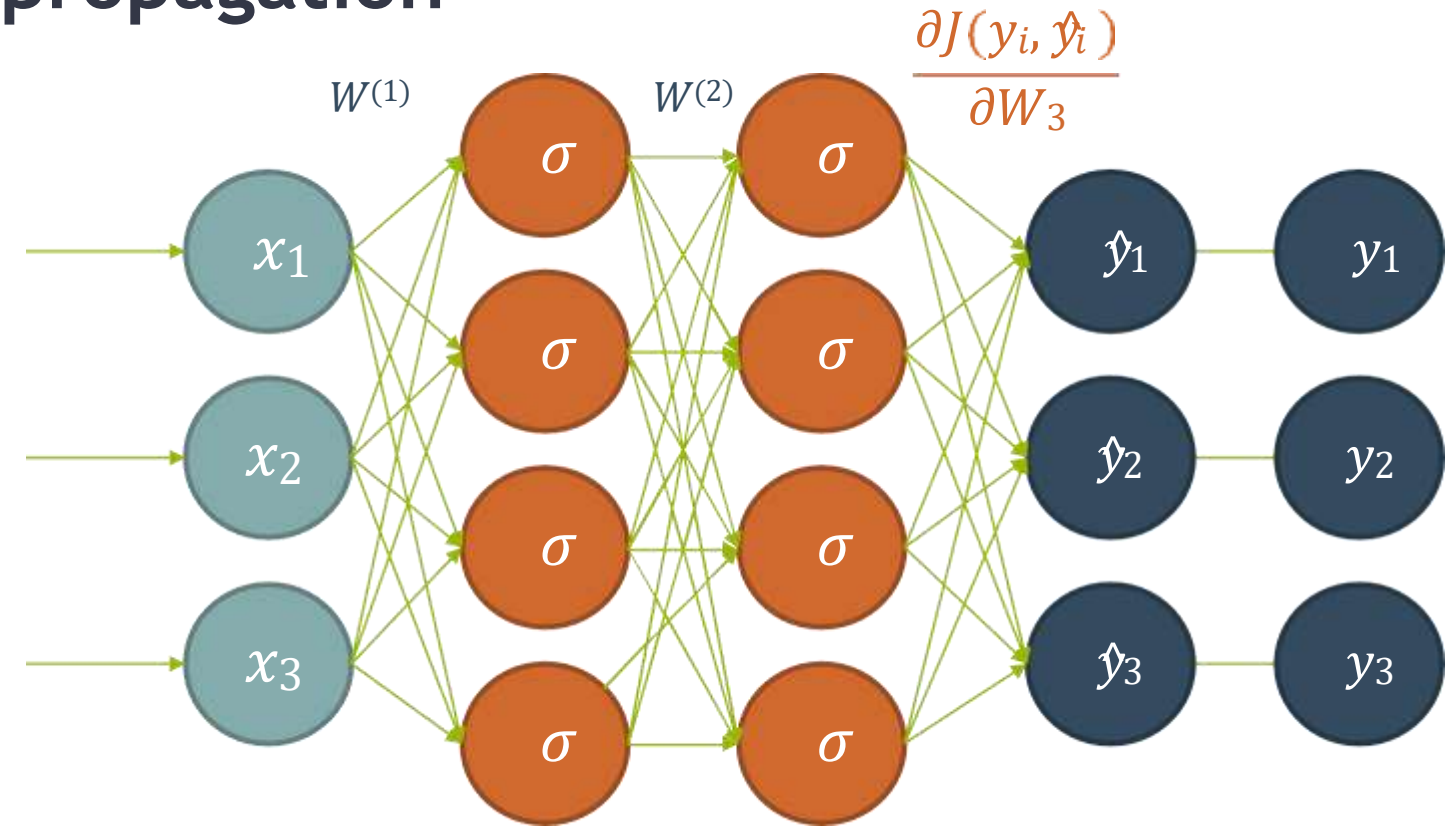
- Though they appear complex, above are easy to compute!

Backpropagation

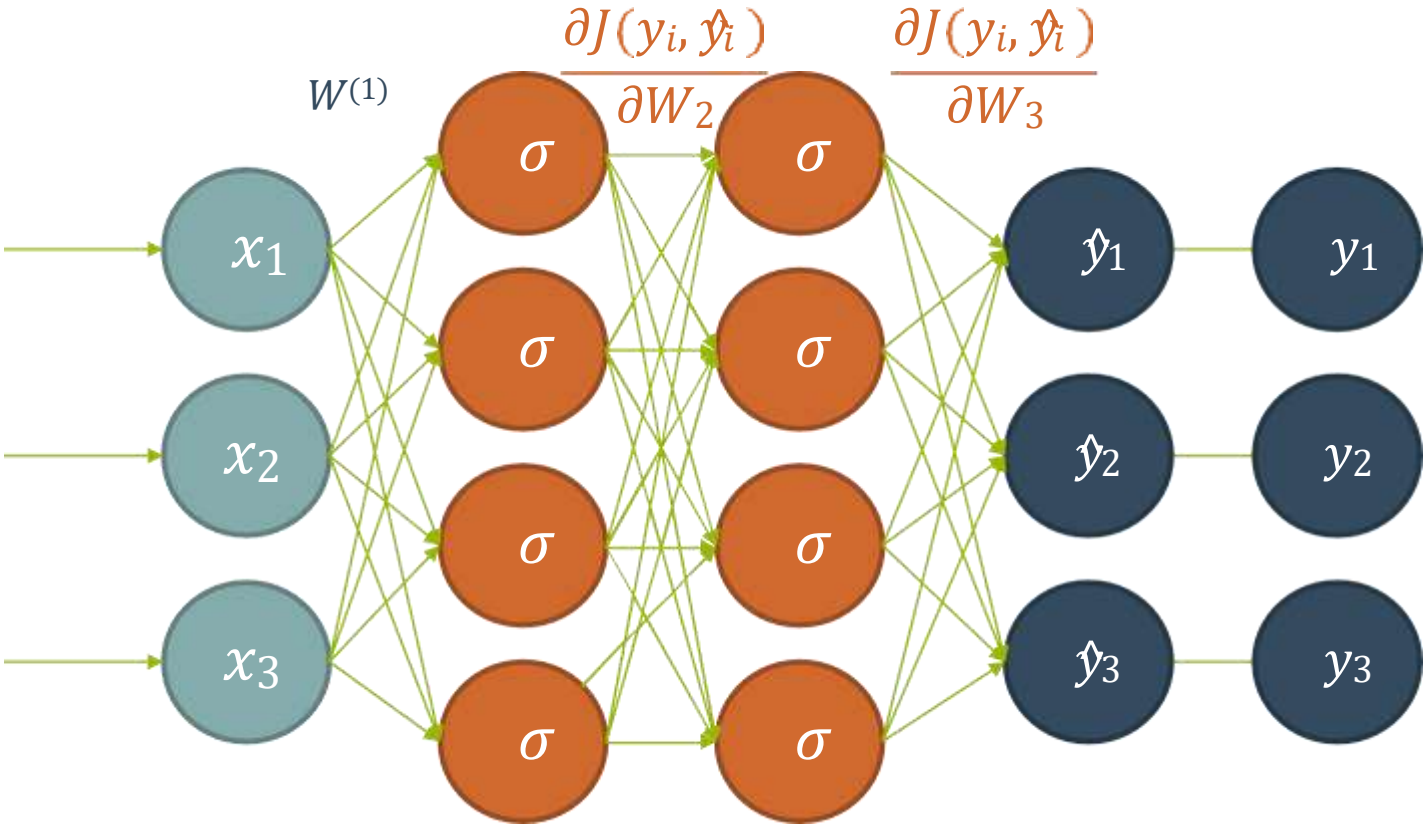
Want: $\frac{\partial J(y_i, \hat{y}_i)}{\partial W_k}$



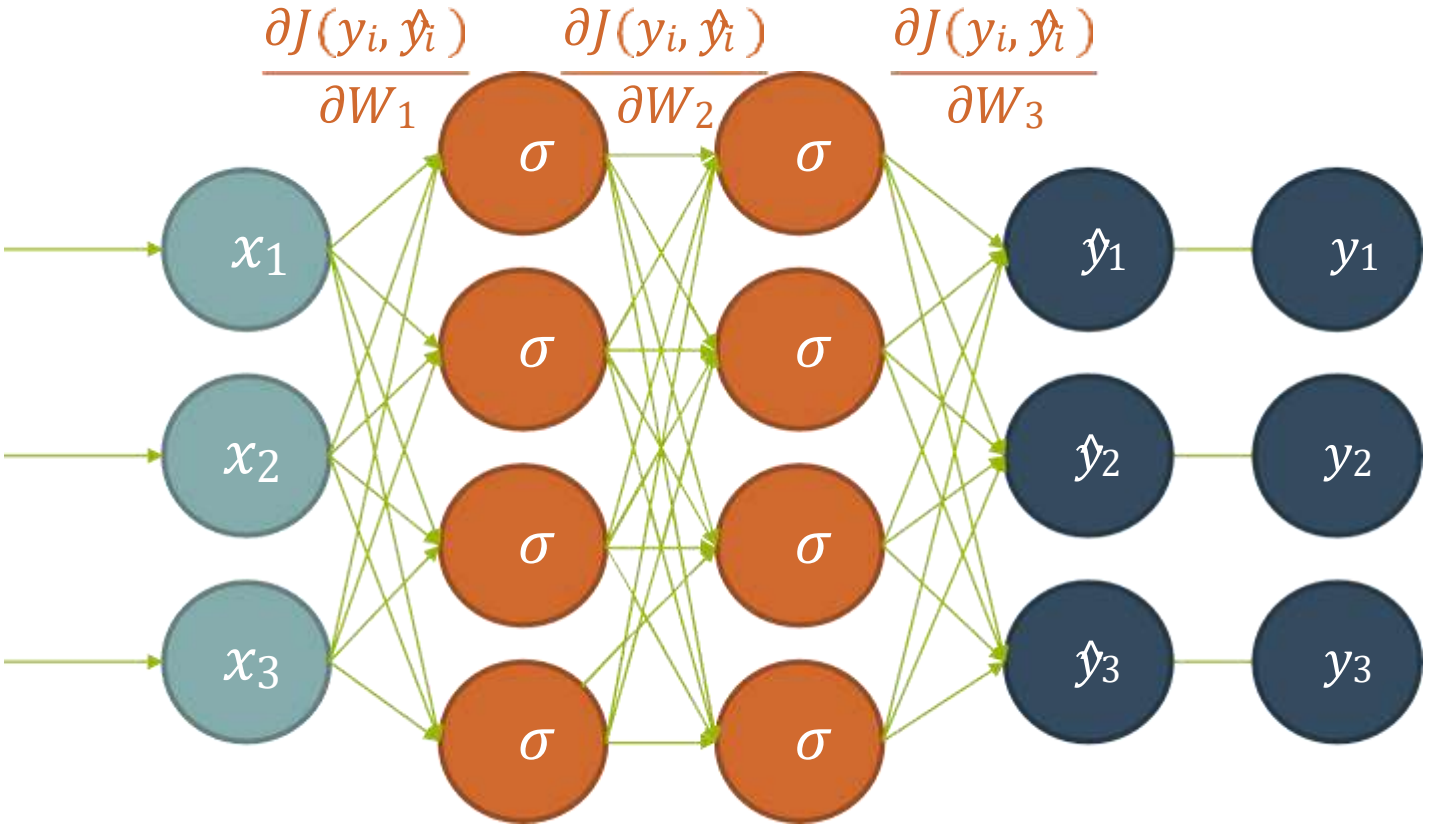
Backpropagation



Backpropagation



Backpropagation



How have we trained before?

Gradient Descent!

1. Make prediction
2. Calculate Loss
3. Calculate gradient of the loss function w.r.t. parameters
4. **Update parameters by taking a step in the opposite direction**
5. Iterate

Other Activation Functions

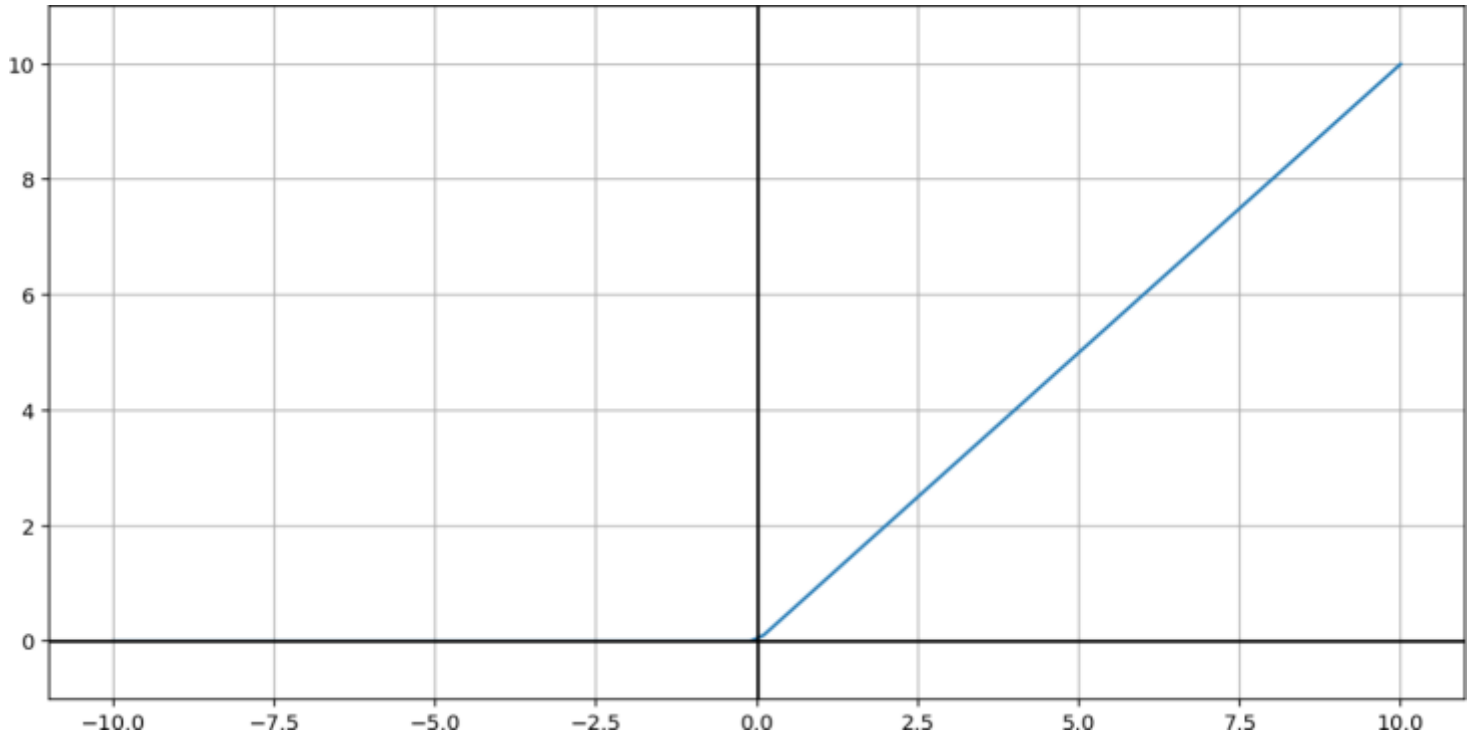
Rectified Linear Unit (ReLU)

$$ReLU(z) = \begin{cases} 0, & z < 0 \\ z, & z \geq 0 \end{cases}$$

$$= \max(0, z)$$

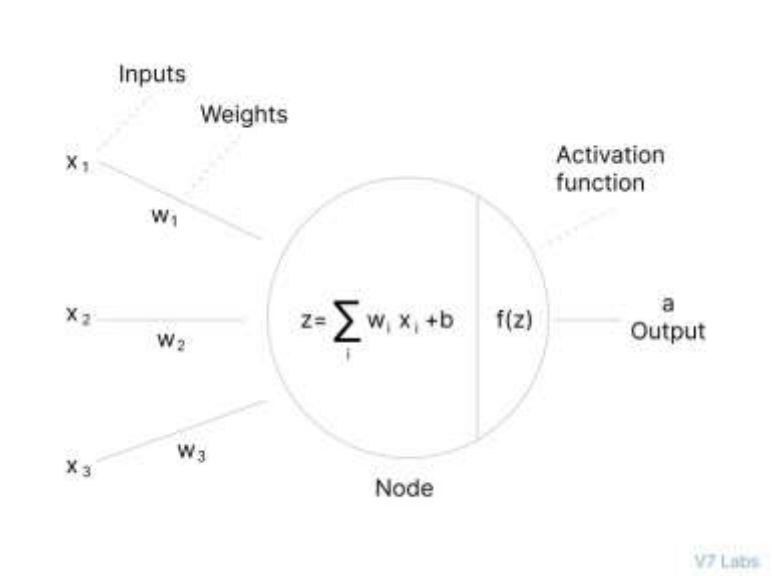
$$ReLU(0) = 0$$
$$ReLU(z) = z \qquad \text{for } (z \gg 0)$$
$$ReLU(-z) = 0$$

Rectified Linear Unit (ReLU)



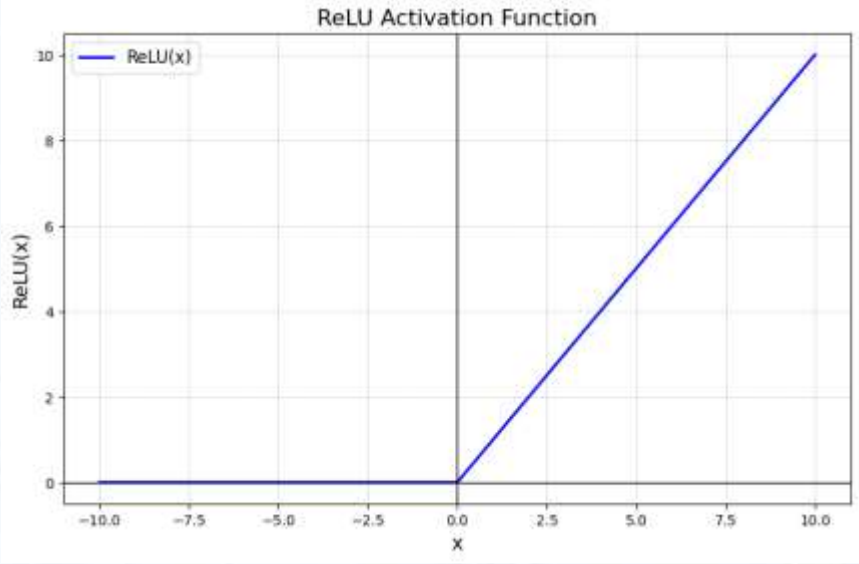
Activation functions

Introduce **nonlinearity** so networks approximate complex functions.



Some Activation functions

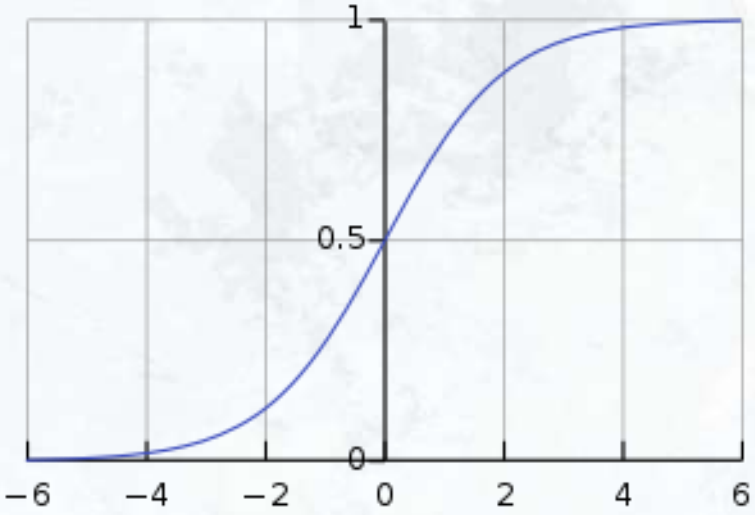
$$R(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$



ReLU

Some Activation functions

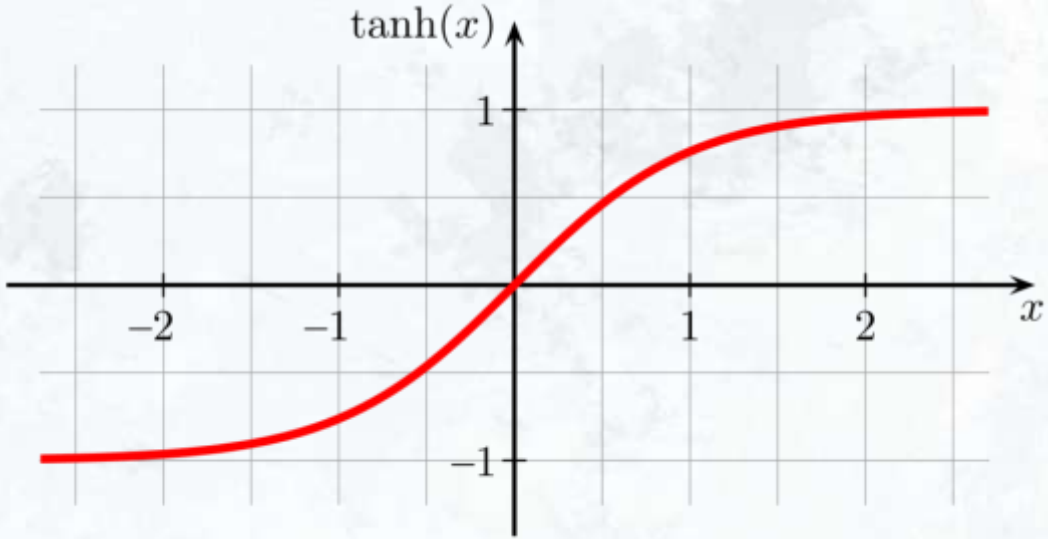
$$f(x) = \frac{1}{1+e^{-(x)}}$$



Sigmoid

Some Activation functions

$$f(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$



tanh

DETAILS OF TRAINING NEURAL NETS

What next?

- Given an example (or group of examples), we know how to compute the derivative for each weight.
- How exactly do we update the weights?
- How often? (after each training data point? after all the training data points?)

What next?—Gradient Descent

- W_new =W_old - lr *derivative
- Classical approach—get derivative for entire data set, then take a step in that direction
- Pros: Each step is informed by all the data
- Cons: Very slow, especially as data gets big

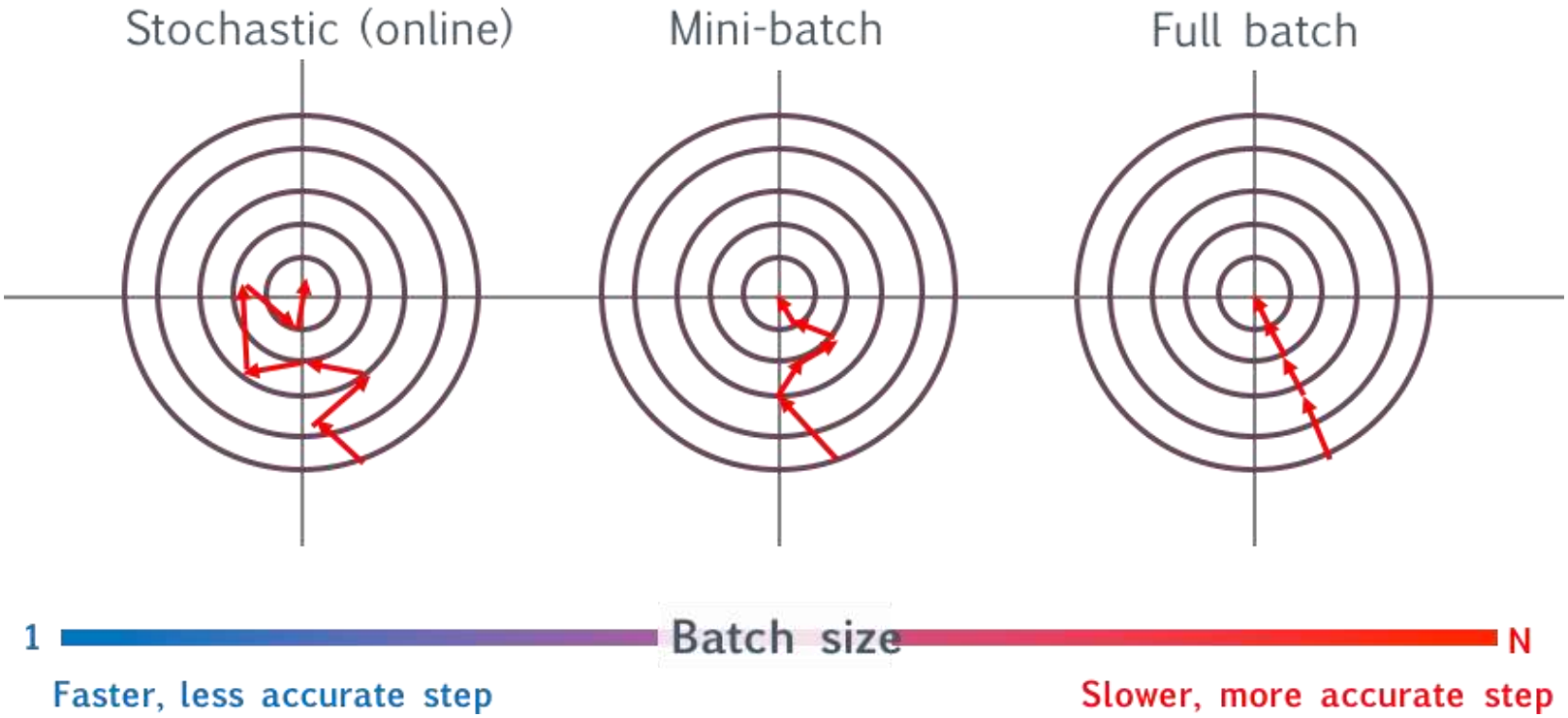
Another approach: Stochastic Gradient Descent

- Get derivative for just one point, and take a step in that direction
- Steps are "less informed" but you take more of them
- Should "balance out"
- Probably want a smaller step size
- Also helps "regularize"

Compromise approach: Mini-batch

- Get derivative for a "small" set of points, then take a step in that direction
- Typical mini batch sizes are 16, 32
- Strikes a balance between two extremes

Comparison of Batching Approaches



Batching Terminology

- Full-batch:
- Use entire data set to compute gradient before updating

- Mini-batch:
- Use a smaller portion of data (but more than single example) to compute gradient before updating

- Stochastic Gradient Descent (SGD):
- Use a single example to compute gradient before updating (though sometimes people use SGD to refer to minibatch, also)

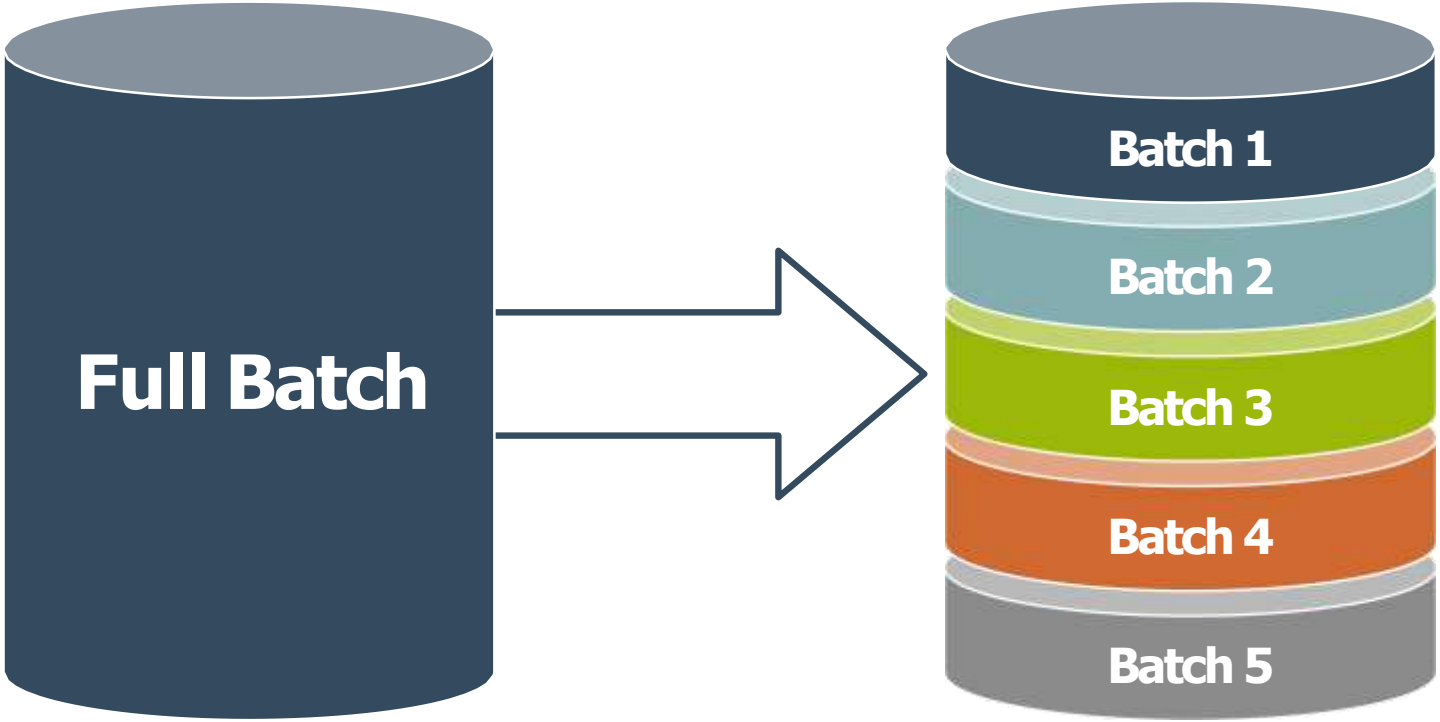
Batching Terminology

- An **Epoch** refers to a single pass through all of the training data.
- In full batch gradient descent, there would be one step taken per epoch.
- In SGD / Online learning, there would be n steps taken per epoch (n = training set size)
- In Minibatch there would be (n/batch size) steps taken per epoch
- When training, it is common to refer to the number of epochs needed for the model to be "trained".

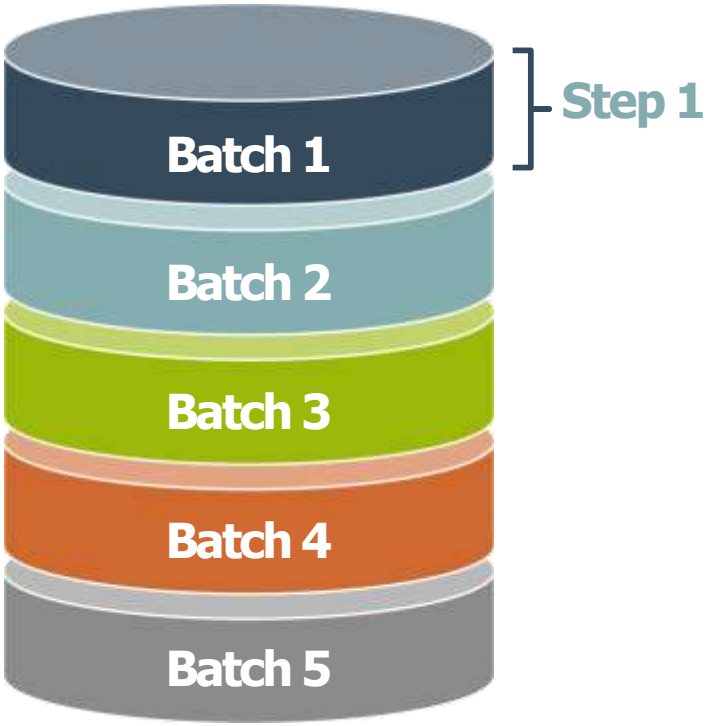
Note on Data Shuffling

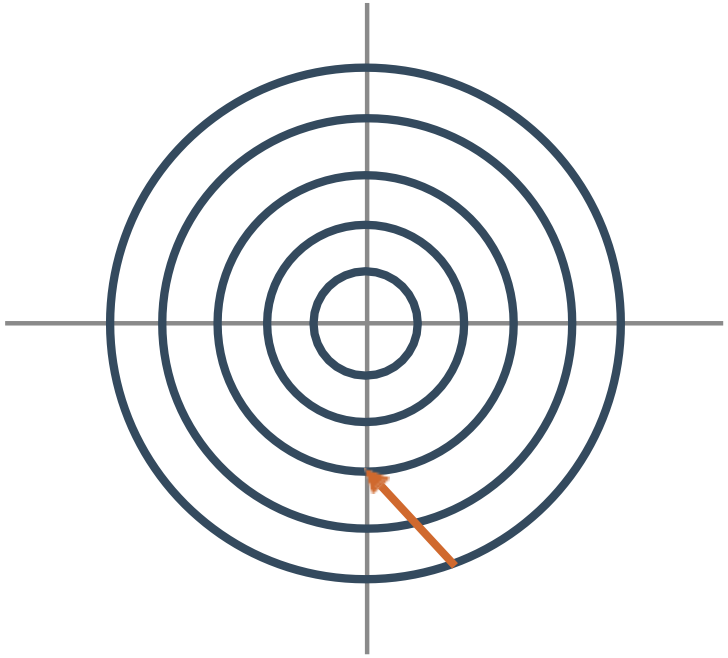
- To avoid any cyclical movement and aid convergence, it is recommended to shuffle the data after each epoch.
- This way, the data is not seen in the same order every time, and the batches are not the exact same ones.

Feedforward Neural Network

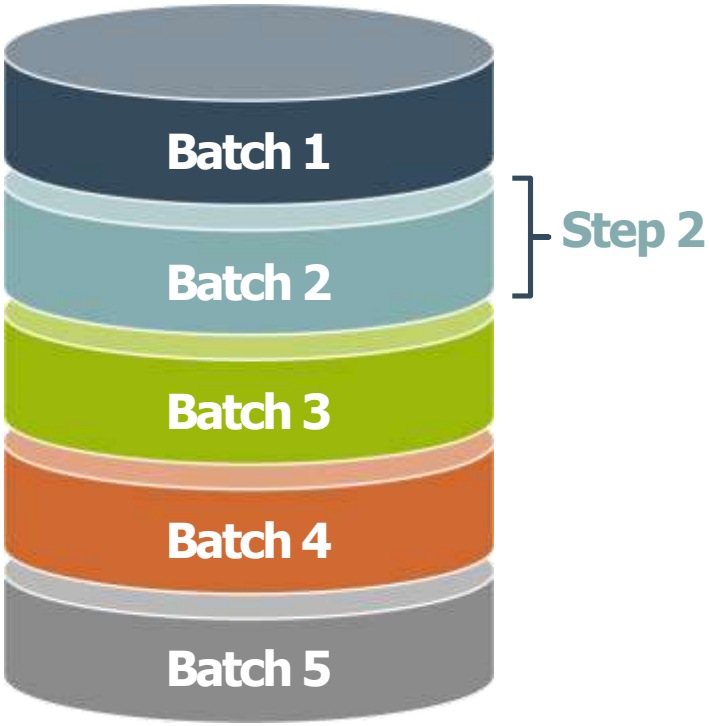


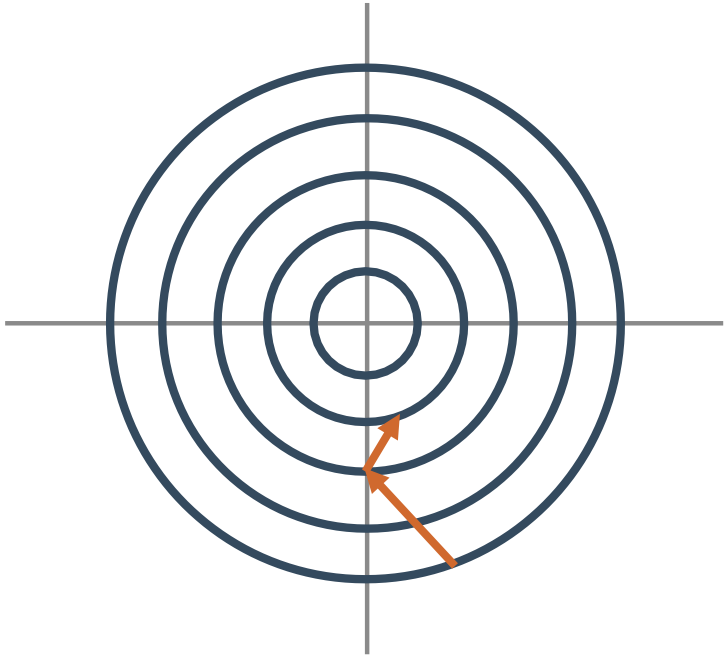
Training in Action



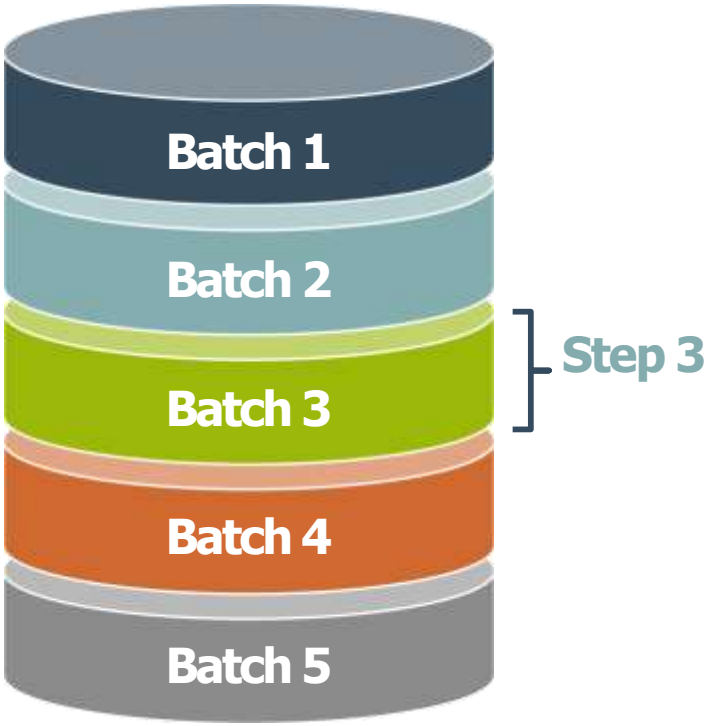


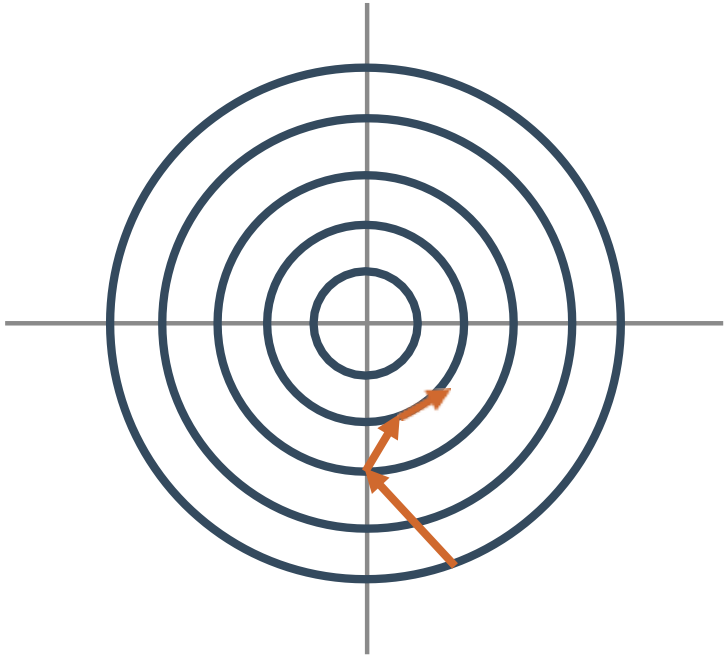
Training in Action



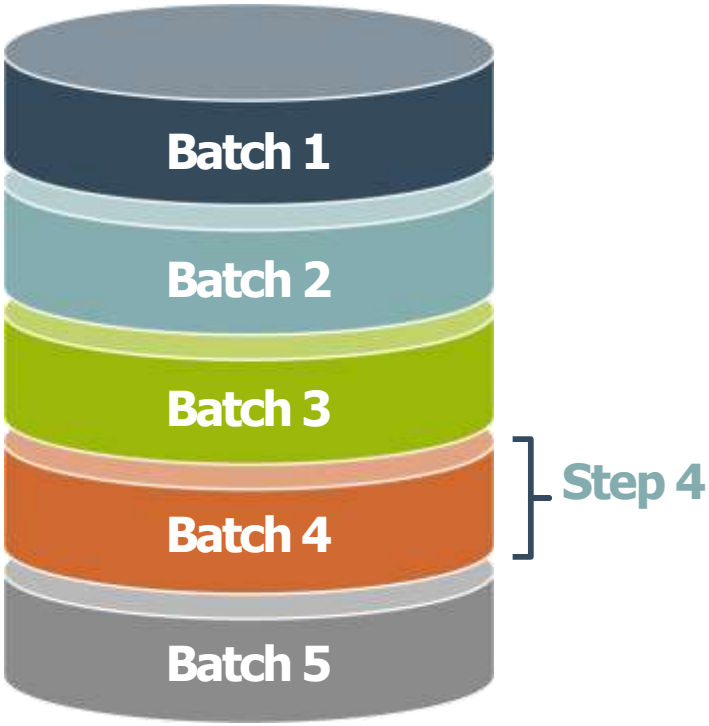


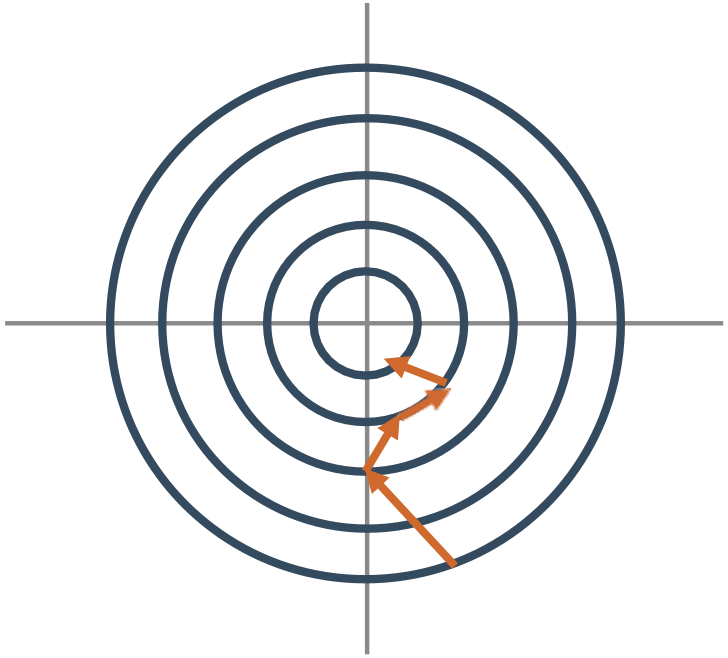
Training in Action



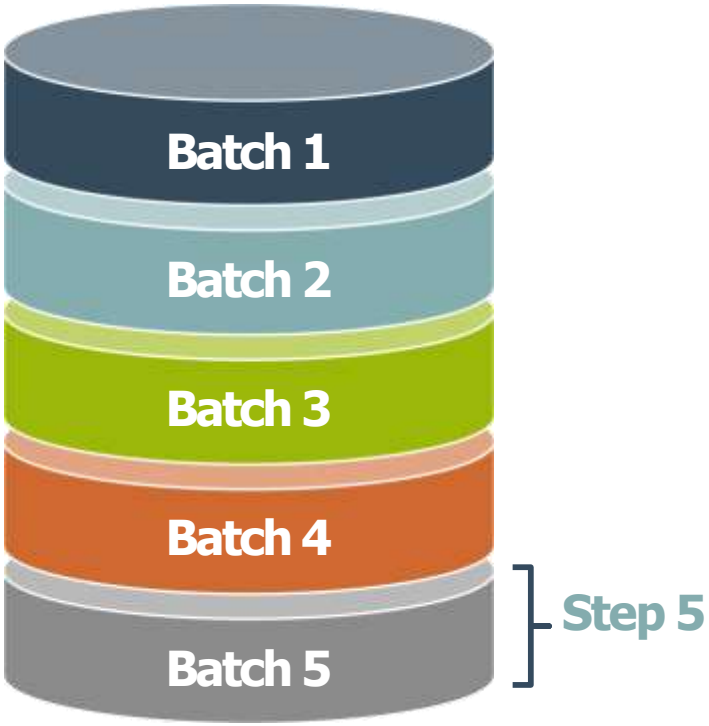


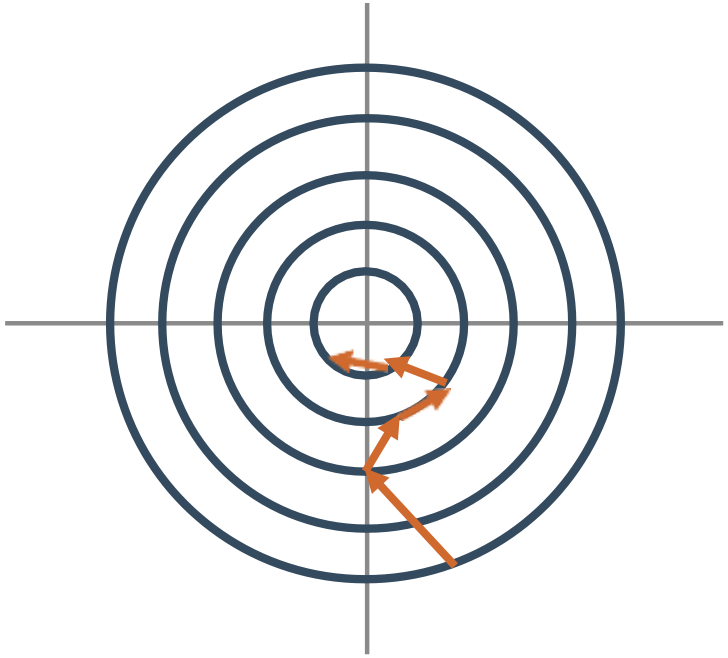
Training in Action



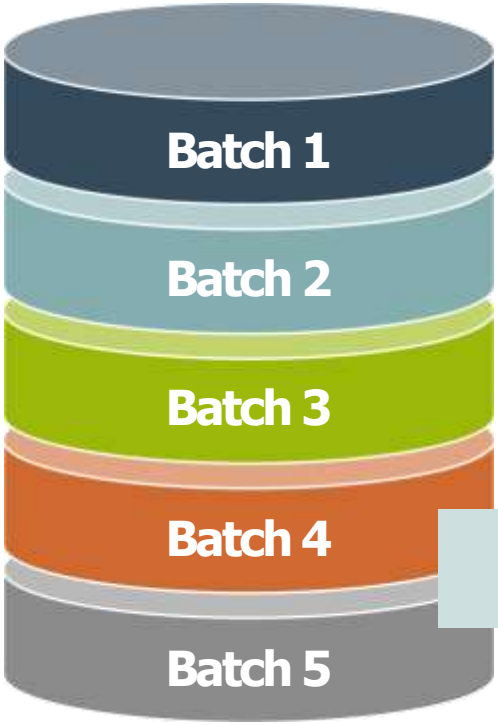


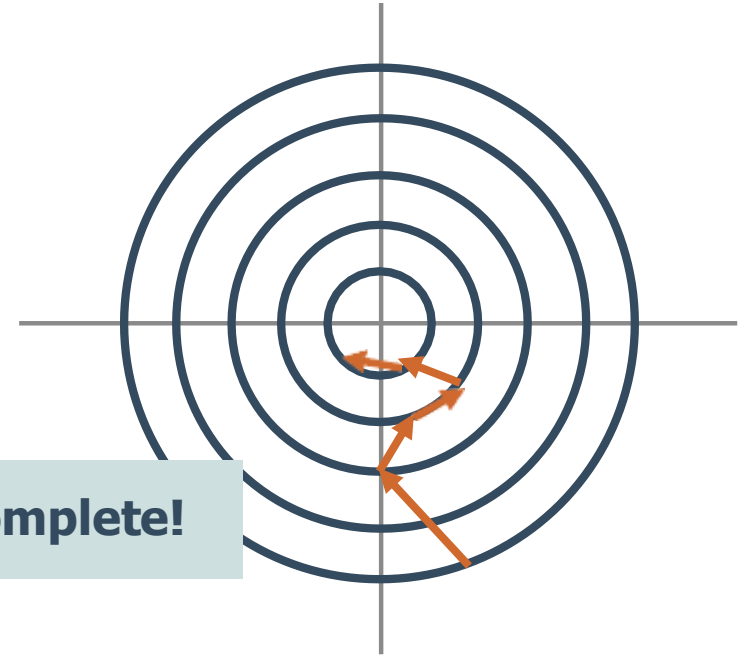
Training in Action





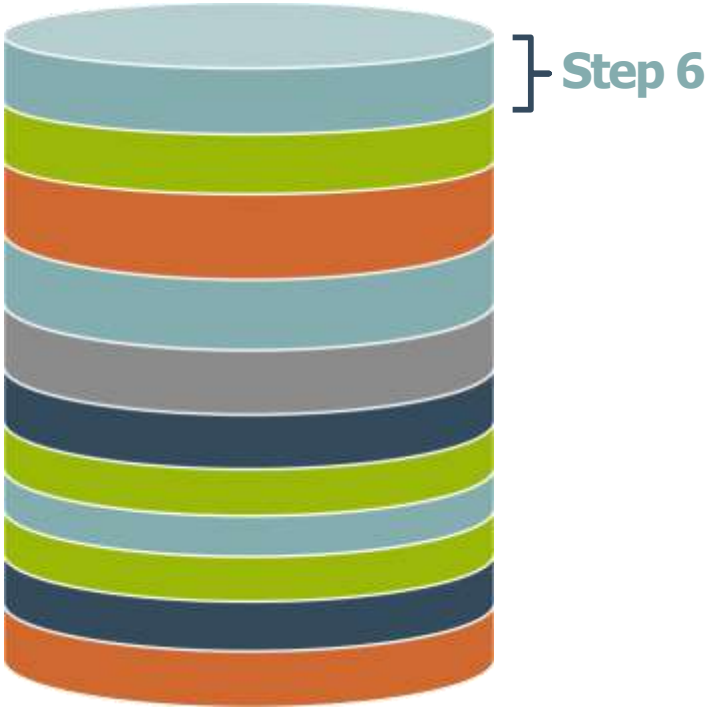
Training in Action

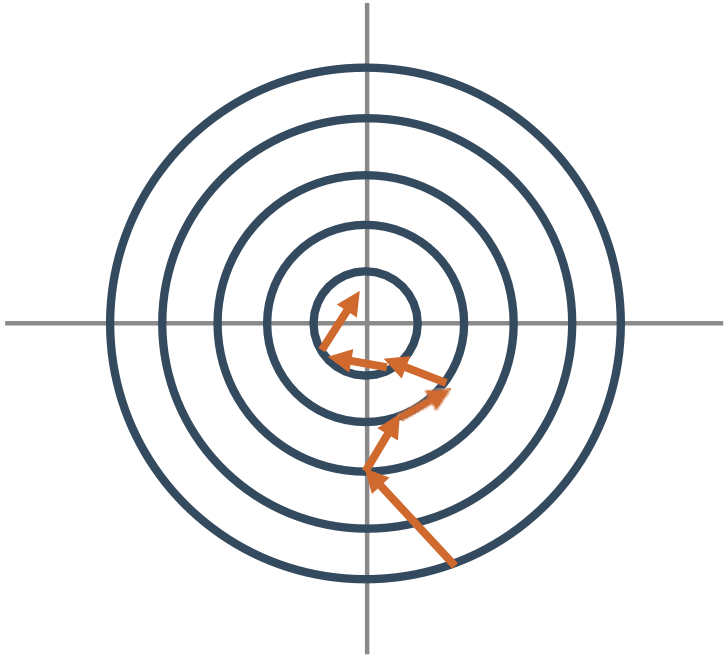




First Epoch Complete!

Training in Action





Shuffle the Data!

The Keras Package

- Keras allows easy construction, training, and execution of Deep Neural Networks
- Written in Python, and allows users to configure complicated models directly in Python
- Uses either Tensorflow or Theano "under the hood"
- Uses either CPU or GPU for computation
- Uses numpy data structures, and a similar command structure to scikit-learn (model.fit , model.predict, etc.)

Typical Command Structure in Keras

- Build the structure of your network.
- Compile the model, specifying your loss function, metrics, and optimizer (which includes the learning rate).
- Fit the model on your training data (specifying batch size, number of epochs)
- Predict on new data
- Evaluate your results

Building the model

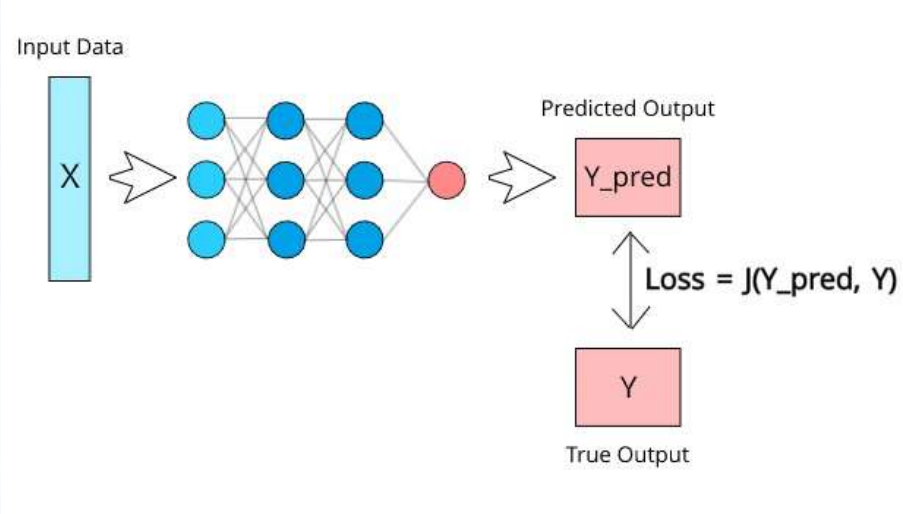
- Keras provides two approaches to building the structure of your model:
- Sequential Model: allows a linear stack of layers – simpler and more convenient if model has this form
- Functional API: more detailed and complex, but allows more complicated architectures
- We will focus on the Sequential Model.

Why neural networks over classical machine learning ?

- **Automated feature learning:** Classical models (e.g., linear/logistic regression, …) need manual **feature engineering**. NNs learn features directly from raw data
- **Handling unstructured data:** Deep NNs excel on **images, audio, and text**, where structured feature design is hard.
- **Scalability:** With more data, deeper networks tend to improve, unlike many classical algorithms that plateau.

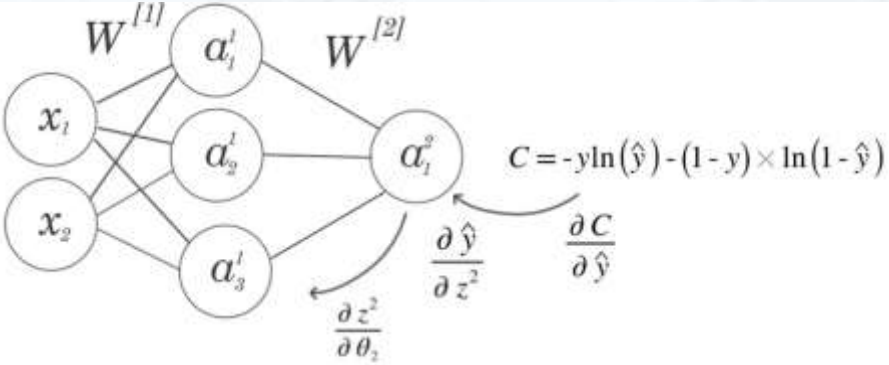
Loss Function

- **Mean Squared Error (MSE)** for regression.
- **Cross-entropy** for classification.



Backpropagation

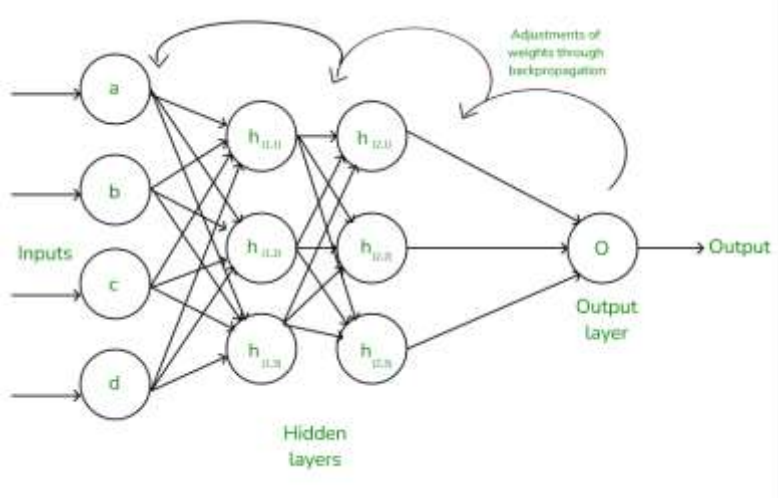
- Computes gradients of the loss. each weight using the chain rule, updating weights via gradient descent
- Foundation for virtually all NN training algorithms.



$$\frac{\partial C}{\partial \theta_2} = \frac{\partial z^2}{\partial \theta_2} \frac{\partial \hat{y}}{\partial z^2} \frac{\partial C}{\partial \hat{y}}$$

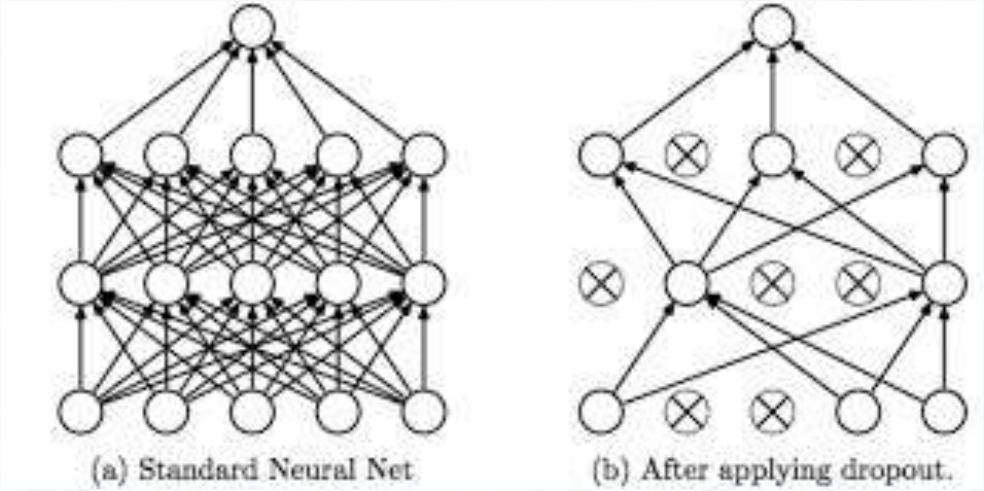
Backpropagation

- Computes gradients of the loss. each weight using the chain rule, updating weights via gradient descent
- Foundation for virtually all NN training algorithms.



Dropout

- randomly “drops” units during training to prevent co-adaptation and **overfitting**
- Acts like ensemble averaging of thinned networks.



02 →

TensorFlow

(AI)

Tensorflow

- An **open-source** machine learning framework developed by **Google.**
- Used for building and training models (including **deep neural networks**).
- Supports low-level operations for maximum control and performance.
- Works efficiently on **CPU, GPU, and TPU** environments.

Keras

- A **high-level API** built on top of **TensorFlow.**
- Simplifies building **neural networks** with an intuitive, user-friendly syntax.
- Ideal for beginners and prototyping.
- Fully integrated into TensorFlow as *tf.keras.*

TensorFlow VS Keras

Deep learning development: layers, models, optimizers, losses, metrics...

Tensor manipulation infrastructure: tensors, variables, automatic differentiation, distribution...

Hardware: execution

InputLayer

- Defines the input shape of the model

```
keras.layers.InputLayer(input_shape=(28, 28))
```

Dense

- Fully connected layer where each neuron is connected to all neurons in the previous layer.

```
keras.layers.Dense(128, activation='relu')
```

Dropout

- Randomly sets a fraction of input units to zero during training.

```
keras.layers.Dropout(0.2)
```

Sequential.compile()

- Configures the model for training.
- ***optimizer:*** Defines how the model updates weights during training.
- ***loss:*** loss function
- ***metrics*:** Specifies what to monitor during training and evaluation.

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Sequential.fit()

- Trains the model on your data.

```
model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

Sequential.evaluate()

- Tests the model on unseen data and returns metrics.

```
loss, accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {accuracy}")
```

Sequential.predict()

- Generates predictions (probabilities) from the model.

```
predictions = model.predict(x_test)
print(predictions[0])  # Probabilities for the first test image
```